

Clase 1 - 10 de marzo, 2026

Introducción (historia)

- **Inicios del siglo XX:** Se empieza a buscar un esquema mecánico para demostrar todas las verdades matemáticas (Hilbert).
- **1931:** Gödel demuestra que hay verdades matemáticas que no pueden ser demostradas mecánicamente.
- **1936:** Church y Turing demuestran que hay enunciados matemáticos que no se puede decidir mecánicamente si son verdaderos o falsos (problema de la parada). Origen de la computación. Máquina de Turing como modelo de computación.
- **Década de los 40:** Primeras computadoras.
- **Década de los 60:** Origen de la complejidad computacional, problemas fáciles y difíciles en términos de tiempo y espacio. Algoritmos probabilísticos y cuánticos, pruebas interactivas, inteligencia artificial, etc.
- **Década de los 70:** Origen de la correctitud de programas (verificación formal, semántica de lenguajes). Evoluciona a la verificación semi-automática (con asistencia) y la verificación automática (model checking), etc.

Repaso de conceptos básicos de matemática

Conjunto

- Un conjunto es una colección de elementos.
- $A = \{a, b, c\}$ es un conjunto con tres elementos: a , b y c . Se dice que a , b y c pertenecen (o son miembros de) A , lo que se denota como $a \in A$, $b \in A$ y $c \in A$.

Igualdad de conjuntos

- Dos conjuntos A y B son iguales, denotado como $A = B$, si tienen los mismos elementos.
- $A = \{casa, árbol, cielo\}$ y $B = \{árbol, cielo, casa\}$ son iguales porque contienen los mismos elementos, aunque estén en diferente orden.

Subconjunto

- Un conjunto A es un subconjunto de otro conjunto B , denotado como $A \subseteq B$, si todos los elementos de A también son elementos de B . La inclusión puede ser estricta, denotada como $A \subset B$, si A es un subconjunto de B pero los conjuntos NO son iguales. Si no es estricta, denotada como $A \subseteq B$, entonces A puede ser igual a B .
- $\{1, 2\} \subseteq \{1, 2, 3, 5\}$ y también se cumple que $\{1, 2\} \subset \{1, 2, 3, 5\}$.

Intersección

- Dados dos conjuntos A y B , la intersección de A y B , denotada como $A \cap B$, es el conjunto de elementos que están en A y en B .

- $\{1, 2, 3, 4, 5\} \cap \{4, 5, 6\} = \{4, 5\}$

Unión

- Dados dos conjuntos A y B , la unión de A y B , denotada como $A \cup B$, es el conjunto de elementos que están en A o en B (o en ambos).
- $\{1, 2, 3, 4, 5\} \cup \{4, 5, 6\} = \{1, 2, 3, 4, 5, 6\}$

Diferencia

- Dados dos conjuntos A y B , la diferencia de A y B , denotada como $A - B$, es el conjunto de elementos que están en A y no están en B .
- $\{1, 2, 3, 4, 5\} - \{4, 5, 6\} = \{1, 2, 3\}$

Funciones

- Una función de un conjunto A a un conjunto B , denotada como $f : A \rightarrow B$, es un conjunto de pares ordenados (a, b) donde $a \in A$ y $b \in B$, tales que a lo sumo existe un par (a, b) para cada $a \in A$. Es decir, cada elemento de A se asigna a lo sumo a un elemento de B .
- $f(a) = b$ expresa que el par ordenado (a, b) pertenece a la función f .
- Por ejemplo:
 - $A = \{1, 2, 3, 4\}$
 - $B = \{m, n, o\}$
 - $\{(1, m), (2, n), (3, n)\}$ es una función f de A a B .
 - También se puede definir así:
 - $f(1) = m$
 - $f(2) = n$
 - $f(3) = n$.
 - El conjunto A es el **dominio** y el conjunto B es el **codominio** de la función f .
- f es **inyectiva** si cada elemento de B es asignado como mucho a un elemento de A . Es decir, no hay dos elementos distintos de A que se asignen al mismo elemento de B .
- f es **suryectiva** si cada elemento de B es asignado a al menos un elemento de A . Es decir, todos los elementos de B son alcanzados por la función f .
- f es **biyectiva** si es inyectiva y suryectiva a la vez.
- En el ejemplo, la función f no es inyectiva porque el elemento n de B es asignado a dos elementos distintos de A (2 y 3), y tampoco es suryectiva porque el elemento o de B no es asignado a ningún elemento de A .

Alfabeto

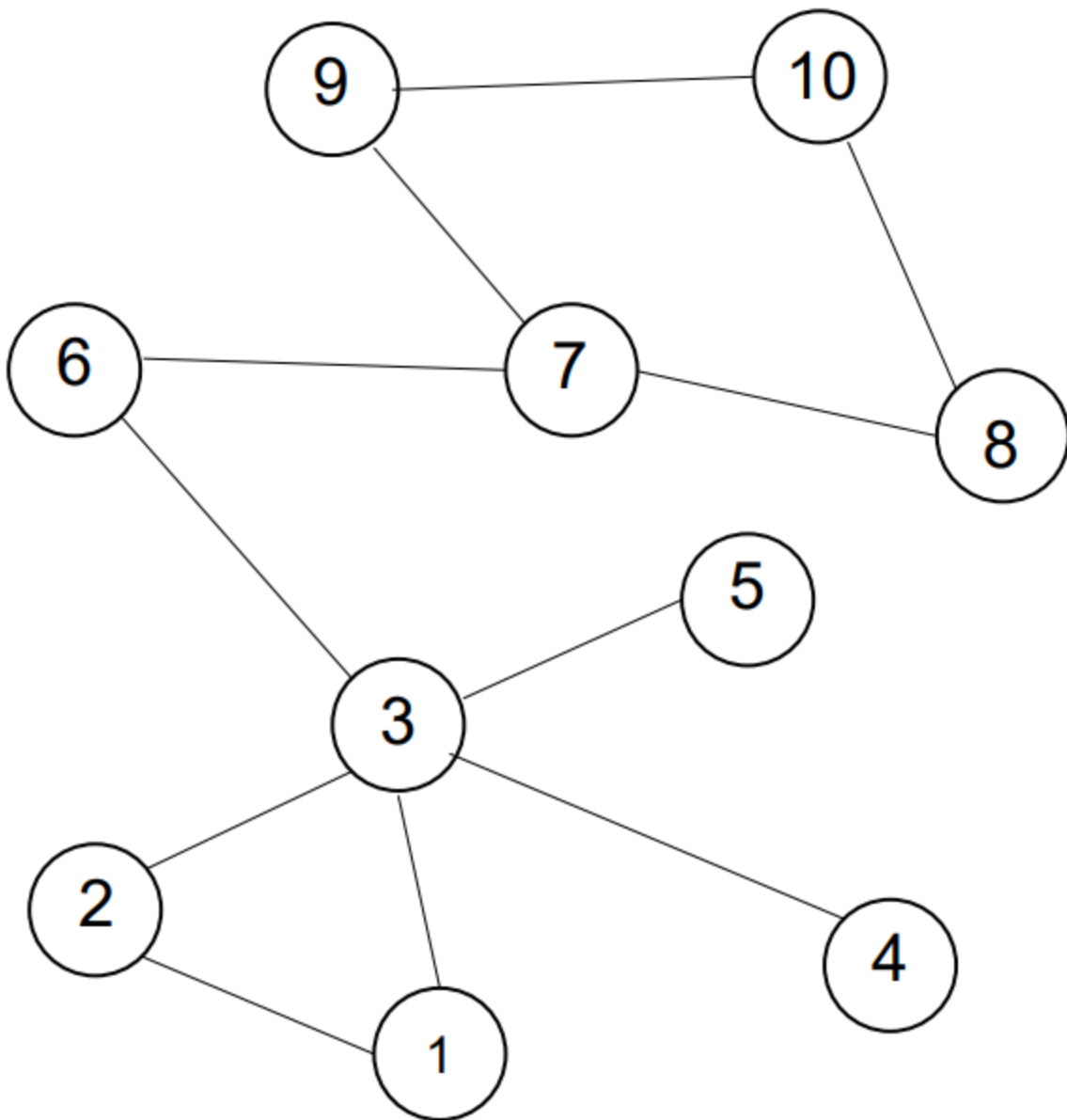
- Un alfabeto es un conjunto finito de símbolos. Se denota comúnmente como Σ .
- $\Sigma = \{a, b, c\}$ es un alfabeto de 3 símbolos.

Lenguaje

- Un lenguaje sobre un alfabeto Σ es un conjunto de cadenas finitas formadas con los símbolos de Σ .
- Si $\Sigma = \{a, b, c\}$, un lenguaje L con alfabeto Σ podría ser $L = \{aaa, b, ababab, ccb\}$.
- Las operaciones típicas entre lenguajes son las mismas que para los conjuntos: intersección, unión y diferencia. Además se agregan:
 - El complemento respecto a otro lenguaje que lo incluye.
 - El complemento respecto a todas las cadenas posibles sobre el alfabeto (denotado como Σ^*).
 - La concatenación de dos lenguajes L_1 y L_2 , denotada como $L_1 \cdot L_2$, que es el lenguaje formado por todas las cadenas que se pueden obtener concatenando una cadena de L_1 con una cadena de L_2 .

Grafo

- Un grafo es un par $G = (V, E)$ donde V es un conjunto de **vértices** y E es un conjunto de **arcos**.



- En este grafo tenemos 10 vértices y 11 arcos.
- Los vértices 1 y 2 son **adyacentes**, es decir, están conectados por un arco.
- Los arcos (1, 2) y (2, 3) son **adyacentes**, es decir, comparten un vértice.
- La secuencia de vértices (1, 2, 3, 6, 7, 9, 10) representa un **camino** del vértice 1 al vértice 10.

Fórmula booleana

- Una fórmula booleana es una fórmula lógica que se obtiene combinando variables booleanas de valor de verdad Verdadero (V) o Falso (F), usando los operadores lógicos AND ($\wedge$), OR ($\vee$) y NOT ($\neg$).
- Por ejemplo $\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$ es una fórmula booleana con variables booleanas x_1 y x_2 .
- La semántica de los operadores lógicos es la siguiente:
 - $\varphi = x_1 \wedge x_2$ es V si y solo si x_1 es V y x_2 es V.
 - $\varphi = x_1 \vee x_2$ es V si y solo si x_1 es V o x_2 es V (o ambos).
 - $\varphi = \neg x$ es V si y solo si x es F.
- Por ejemplo, dada la fórmula booleana $\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$, si $x_1 = V$ y $x_2 = F$, entonces φ es V, pero si $x_1 = V$ y $x_2 = V$, entonces φ es F.
- Una fórmula booleana φ es satisfacible si existe al menos una asignación de valores de verdad que la hace verdadera.
 - Por ejemplo, la fórmula anterior es satisfacible porque existe una asignación de valores de verdad (por ejemplo, $x_1 = V$ y $x_2 = F$) que la hace verdadera.
 - En cambio, la fórmula $\varphi = x_1 \wedge \neg x_1$ no es satisfacible porque no existe ninguna asignación de valores de verdad que la haga verdadera.

Máquina de Turing

Concepto

- Modelización muy simple de una computadora.
- Está aceptada universalmente para estudiar la computabilidad y la complejidad computacional de los problemas.

Cinta

- La máquina de Turing tiene una cinta infinita dividida en celdas.
- Cada celda almacena un único símbolo del alfabeto de la máquina, denotado como Γ .
- La unidad de control, también llamada cabezal, siempre se encuentra en un estado determinado de un conjunto finito de estados denotado como Q .
- Al iniciar la computación, la entrada se delimita por símbolos blancos B a ambos lados, y el cabezal apunta al primer símbolo de la entrada.

Definición formal

- Una Máquina de Turing es una tupla $M = (Q, \Gamma, \delta, q_0, q_A, q_R)$ donde:
 - Q es el conjunto finito de **estados** de M .
 - Γ es el **alfabeto** de M , el cual incluye al símbolo blanco B . Las cadenas de entrada no pueden contener el símbolo blanco.

- δ es la **función de transición** de M que especifica su comportamiento. Dado un estado corriente de Q y un símbolo corriente de Γ , la máquina:
 - Pasa eventualmente a un nuevo estado de Q .
 - Modifica eventualmente el símbolo corriente de Γ .
 - Se mueve un lugar a la izquierda, a la derecha o se queda en el mismo lugar.
- q_0 es el estado **inicial** de M .
- q_A es el estado de **aceptación** de M , en el cual la máquina para.
- q_R es el estado de **rechazo** de M , en el cual la máquina para.

Problemas de decisión vs de búsqueda

- Un problema de búsqueda es un problema que requiere encontrar una solución específica. No son muy interesantes en cuanto a las máquinas de Turing.
- Un problema de decisión es un problema que tiene una respuesta de sí o no. Por ejemplo, dado un número entero n , determinar si n es par o no. Estos problemas son los que se suelen estudiar con máquinas de Turing.
 - Dada una cadena de entrada, un "sí" equivale a que la máquina de Turing acepte la cadena, es decir, que eventualmente llegue al estado de aceptación q_A .
 - Dada una cadena de entrada, un "no" equivale a que la máquina de Turing rechace la cadena, es decir, que eventualmente llegue al estado de rechazo q_R .
- Dado lo anterior, una MT acepta o reconoce un lenguaje: el lenguaje formado por las cadenas que la máquina acepta. Por ejemplo, si una MT acepta todas las cadenas que representan números enteros pares, entonces el lenguaje que reconoce es el lenguaje de los números enteros pares. Se denota con $L(M)$ al lenguaje reconocido por la máquina de Turing M .
- Las palabras **problema** y **lenguaje** se usan indistintamente.

Problemas computables decidibles

- Se trata de problemas de decisión donde dada una cadena de entrada w , la máquina de Turing M **siempre se detiene** (es decir, llega eventualmente a un estado de aceptación o de rechazo) y responde correctamente si w pertenece o no al lenguaje que reconoce M .
- Ejemplo: el problema de decidir si un número entero es par o no es un problema computable decidable, porque existe una máquina de Turing que siempre se detiene y responde correctamente si la cadena de entrada representa un número entero par o no.

Problemas computables no decidibles

- Se trata de problemas de decisión donde dada una cadena de entrada w , la máquina de Turing M **no siempre se detiene**, porque si bien siempre llega al estado de aceptación para las cadenas que sí pertenecen al lenguaje de la máquina, esto no se cumple para las cadenas que no pertenecen al lenguaje, porque la misma puede **loopear** indefinidamente.
- Ejemplo: el problema de la parada es un problema computable no decidable, porque existe una máquina de Turing que acepta todas las cadenas que representan programas que se detienen, pero no existe una máquina de Turing que rechace todas las cadenas que representan programas que no se detienen, porque la misma puede loopear indefinidamente.

Problemas no computables

- Se trata de problemas de decisión que no poseen ninguna MT que los resuelva, es decir, ni siquiera hay garantía de que las cadenas que sí pertenecen al lenguaje de la máquina sean aceptadas, porque la máquina puede looppear indefinidamente tanto para las cadenas válidas como para las inválidas.
- Ejemplo: el problema de decidir si dos programas son equivalentes o no.

Subdivisión de los problemas computables decidibles

- La principal clasificación de los problemas computables decidibles se hace en base a su **complejidad**:
 - **Tratables**: problemas que pueden ser resueltos por una máquina de Turing en tiempo polinomial respecto al tamaño de la cadena de entrada.
 - **Intratables**: problemas que no pueden ser resueltos por una máquina de Turing en tiempo polinomial respecto al tamaño de la cadena de entrada, sino que requieren un tiempo exponencial o incluso peor.

Varias cintas

- Se trata de una variante de la máquina de Turing que tiene más de una cinta, cada una con su propio cabezal.
- La primer cinta de todas es la que contiene a la cadena de entrada. Las demás empiezan completamente vacías.
- La MT puede, en un solo paso, modificar el estado corriente, los símbolos corrientes de todas las cintas, y moverse distinto (o no) en cada cinta.
- Teorema: para toda MT de varias cintas, existe una MT de una sola cinta que reconoce el mismo lenguaje. Es decir, las MT de varias cintas no poseen más potencia computacional que las MT de una sola cinta, aunque sí pueden ser más eficientes en términos de tiempo de ejecución.

Modelos equivalentes de MT

- Dos MT son equivalentes si aceptan el mismo lenguaje, es decir, si resuelven el mismo problema.
- Dos modelos de MT son equivalentes si dada una MT de un modelo existe una MT equivalente en el otro modelo.
- Ejemplos de modelos de MT equivalentes al modelo de MT clásico de una sola cinta:
 - MT de varias cintas.
 - MT con cintas semi-infinitas (solo infinitas a derecha o a izquierda).
 - MT con dos cintas y un único estado.
- Ejemplos de modelos computacionales equivalentes al modelo de las MT:
 - Máquinas RAM (acceso directo).
 - Circuitos booleanos.
 - Cálculo lambda.
 - Funciones recursivas.
 - Gramáticas.
 - Programas C.
- Lo anterior refuerza la tesis de Church-Turing.

Lenguaje Σ^*

- Dado un alfabeto Σ , el lenguaje Σ^* es el lenguaje formado por todas las cadenas finitas posibles que se pueden formar con los símbolos de Σ .
- Su primer elemento siempre es la cadena vacía λ , que no contiene ningún símbolo.
- Es un conjunto infinito contable.
- Cada uno de sus elementos es una cadena **finita**, por más larga que sea.
- Todo lenguaje L es un subconjunto de Σ^* , es decir, $L \subseteq \Sigma^*$.

MT calculadora

- Problema:
 - Construir una MT que reste dos números codificados en unario (es decir, el número n se codifica como una cadena de n símbolos iguales, por ejemplo, el número 3 se codifica como "111") y separados por un cero.
 - Por ejemplo, dada la entrada 11111101111 hay que obtener la salida 11 porque $6 - 4 = 2$.
- Solución: tachar el primer 1 **antes** del 0, luego el primer 1 **después** del 0, luego el segundo 1 antes del 0, luego el segundo 1 después del 0, y así sucesivamente hasta tachar al final el único 0 de la cadena.

MT generadora

- Se trata de una MT que genera en una cinta de salida todas las cadenas del lenguaje que acepta.
- Por ejemplo, dado el lenguaje $L = \{a^n b^n \mid n \geq 1\}$, la salida debe generar las cadenas $ab, aabb, aaabbb, aaaabbbb$, etc.
- Solución
 - i. $i := 1$
 - ii. Imprimir i veces a , imprimir i veces b , e imprimir una coma
 - iii. $i := i + 1$ y volver al paso 2.
- **Teorema:** existe una MT que acepta un lenguaje L si y solo si existe otra MT que genera el mismo lenguaje L . Es decir, si no es posible construir una MT que genere un lenguaje L , entonces tampoco es posible construir una MT que acepte el mismo lenguaje L , y viceversa.

MT no determinística

- Se trata de una MT que para un mismo par (estado, símbolo) puede responder de más de una manera a la vez.
- Una MTN M acepta una cadena de entrada w si y solo si al menos una **computación** de M acepta a w .
- Para simular una MTN con una MTD hay que recorrer en el peor caso todas sus computaciones. Asumiendo que toda computación hace a lo sumo h pasos, entonces ejecutar secuencialmente todas las computaciones requiere unos c^h pasos, siendo c el grado del no determinismo (retardo exponencial).
- Las MTN no modelizan una computadora real, ya que haría falta un número infinito de procesadores. No son físicamente realizables, pero sirven para simplificar las descripciones de las MTD.

Clase 2 - 17 de marzo, 2026

Clasificación de problemas

Problemas computables decidibles

- Lenguajes aceptados por una MT que siempre para.
- Si $w \in L(M)$, M acepta.
- Si $w \notin L(M)$, M rechaza.
- Se dice que L es **recursivo** o **decidible**.
- R es el conjunto de estos lenguajes.

Problemas computables no decidibles

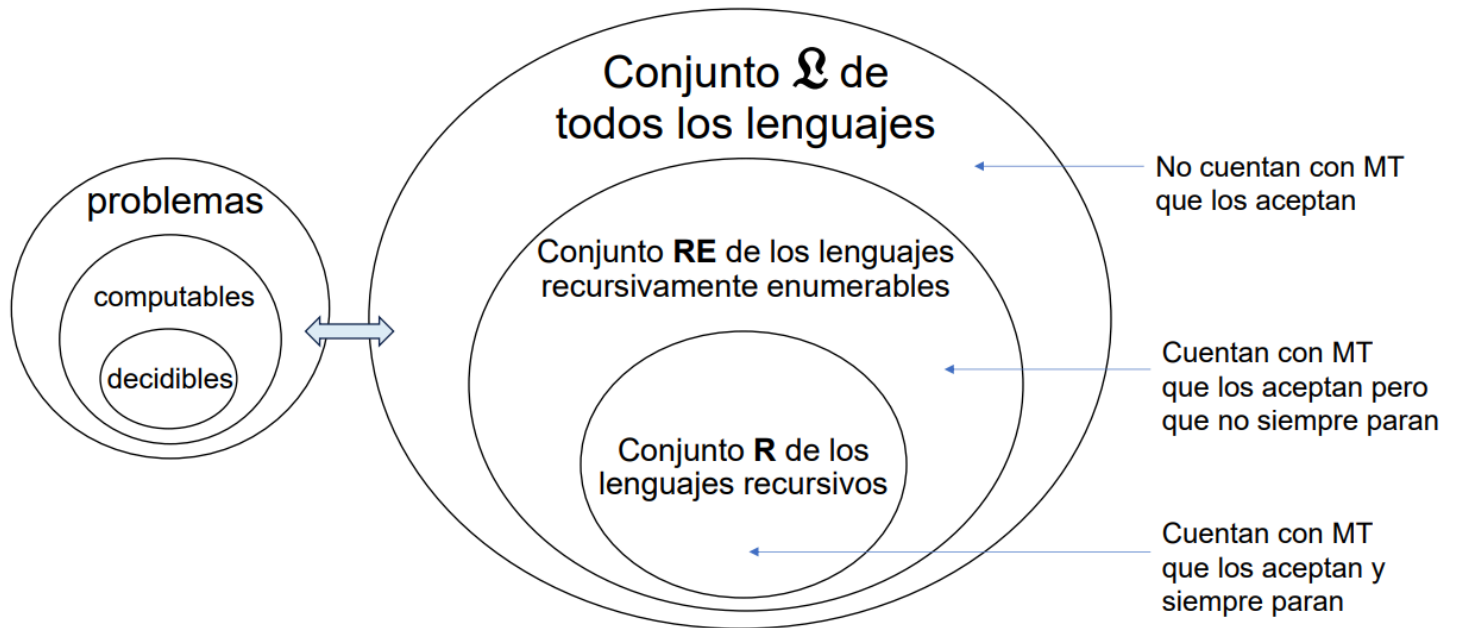
- Lenguajes aceptados por una MT que a partir de algunas instancias negativas no paran.
- Si $w \in L(M)$, M acepta.
- Si $w \notin L(M)$, M rechaza o loopea.
- Se dice que L es **recursivamente enumerable** o **semi-decidible**.
- RE es el conjunto de estos lenguajes.

Problemas no computables

- Lenguajes sin MT que los acepten (a partir de algunas instancias positivas no paran).
- Si $w \in L(M)$, M acepta o loopea.
- Si $w \notin L(M)$, M rechaza o loopea.
- Se dice que L no es **recursivamente enumerable**.

Jerarquía de la computabilidad

Diagrama de Venn



Definiciones formales

- Σ es el **alfabeto** universal de todos los símbolos posibles: $\Sigma = \{a, b, c, \dots, z, 0, 1, 2, \dots, 9, \dots\}$.
- Σ^* es el **conjunto** universal de todas las cadenas de símbolos de Σ : $\Sigma^* = \{\lambda, a, b, c, \dots, z, 0, 1, 2, \dots, 9, aa, ab, ac, \dots\}$.
- $\mathcal{L}$ es el **conjunto** universal de todos los lenguajes sobre Σ : $\mathcal{P}(\Sigma^*)$, es decir, el conjunto de partes de Σ^* .
- Un lenguaje L es **recursivo** ($L \in R$) si y solo si existe una MT M que lo acepta y siempre para (lo decide). Es decir, para toda cadena w de Σ^* :
 - Si $w \in L$, entonces M a partir de w para en el estado q_A .
 - Si $w \notin L$, entonces M a partir de w para en el estado q_R .
- Un lenguaje L es **recursivamente enumerable** ($L \in RE$) si y solo si existe una MT M que lo acepta. Es decir, para toda cadena w de Σ^* :
 - Si $w \in L$, entonces M a partir de w para en el estado q_A .
 - Si $w \notin L$, entonces M a partir de w para en el estado q_R o loopea.
- Se cumple que $R \subset RE \subset \mathcal{L}$, es decir, todo lenguaje recursivo es recursivamente enumerable, pero no todo lenguaje recursivamente enumerable es recursivo, y todo lenguaje recursivamente enumerable es un lenguaje sobre Σ , pero no todo lenguaje sobre Σ es recursivamente enumerable.
 - Las inclusiones son **estrictas**, es decir, $R \subsetneq RE \subsetneq \mathcal{L}$, porque existen lenguajes que son recursivamente enumerables pero no recursivos, y existen lenguajes que no son recursivamente enumerables.

Propiedades de la clase R (lenguajes recursivos)

1. Si $L \in R$, entonces $L^C \in R$.

- i. Es decir, si existe una MT M_1 que decide L , también existe una MT M_2 que decide el complemento de L .
 - ii. Coloquialmente, si un problema es decidible, entonces también lo es el problema opuesto.
 - iii. Se demuestra invirtiendo los estados de aceptación y rechazo de M_1 para construir M_2 . Es decir, M_2 hace exactamente lo mismo que M_1 , pero cuando M_1 acepta, M_2 rechaza, y cuando M_1 rechaza, M_2 acepta. De esta forma M_2 decide el complemento de L .
2. Si $L_1 \in R$ y $L_2 \in R$, entonces $L_1 \cap L_2 \in R$.
- i. Es decir, si existe una MT M_1 que decide L_1 y una MT M_2 que decide L_2 , también existe una MT M_3 que decide la intersección entre L_1 y L_2 .
 - ii. Coloquialmente, si dos problemas son decidibles, entonces también lo es el problema de decidir si ambas condiciones se cumplen a la vez.
 - iii. Se demuestra construyendo una MT M_3 que ejecute secuencialmente M_1 y M_2 y acepte si y solo si ambas aceptan, es decir, M_3 acepta una cadena de entrada w si y solo si M_1 acepta w y luego M_2 acepta w . De esta forma M_3 decide la intersección entre L_1 y L_2 .
3. Si $L_1 \in R$ y $L_2 \in R$, entonces $L_1 \cup L_2 \in R$.
- i. Es decir, si existe una MT M_1 que decide L_1 y una MT M_2 que decide L_2 , también existe una MT M_3 que decide la unión entre L_1 y L_2 .
 - ii. Coloquialmente, si dos problemas son decidibles, entonces también lo es el problema de decidir si al menos una de las condiciones se cumple.
 - iii. Se demuestra construyendo una MT M_3 que ejecute secuencialmente M_1 y M_2 y acepte si al menos una de las dos acepta, es decir, M_3 acepta una cadena de entrada w si y solo si M_1 acepta w o M_2 acepta w . De esta forma M_3 decide la unión entre L_1 y L_2 .

Propiedades de la clase RE (lenguajes recursivamente enumerables)

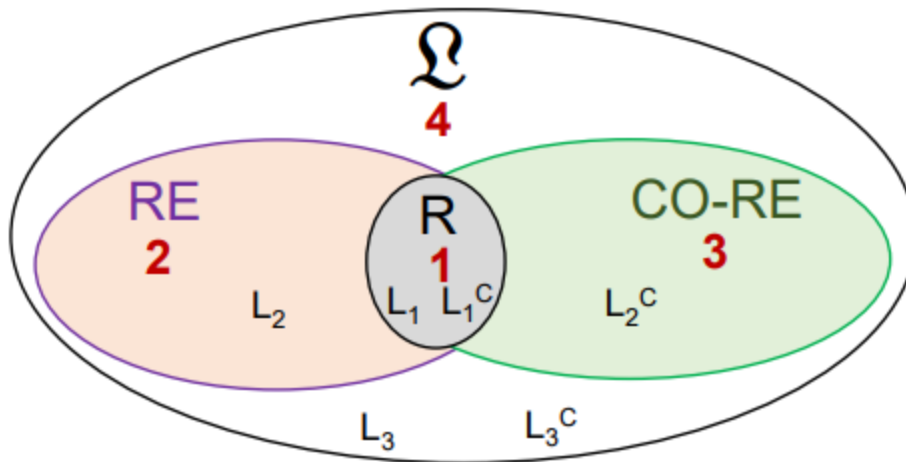
1. Si $L_1 \in RE$ y $L_2 \in RE$, entonces $L_1 \cup L_2 \in RE$.
 - i. Es decir, si existe una MT M_1 que acepta L_1 y una MT M_2 que acepta L_2 , también existe una MT M_3 que acepta la unión entre L_1 y L_2 .
 - ii. Coloquialmente, si dos problemas son semi-decidibles, entonces también lo es el problema de decidir si al menos una de las condiciones se cumple.
 - iii. Para demostrarlo, no se puede hacer lo que se hizo en el caso de los lenguajes recursivos, porque las MT M_1 y M_2 pueden looppear para algunas cadenas de entrada, por lo que no es seguro ejecutar secuencialmente M_1 y M_2 y esperar a que terminen, porque pueden no terminar nunca. Se puede demostrar usando una MT de tres cintas, donde la primera cinta contiene la cadena de entrada, la segunda cinta se usa para simular M_1 y la tercera cinta se usa para simular M_2 . La MT va ejecutando paso por paso las simulaciones de M_1 y M_2 de forma intercalada, es decir, ejecuta un paso de M_1 , luego un paso de M_2 , luego otro paso de M_1 , luego otro paso de M_2 , y así sucesivamente. De esta forma, si al menos una de las dos MT acepta la cadena de entrada, entonces la MT M_3 también la acepta, porque eventualmente llegará a ese paso en el que una de las dos MT acepte.
2. Si $L_1 \in RE$ y $L_2 \in RE$, entonces $L_1 \cap L_2 \in RE$.

La clase CO-RE

- El conjunto CO-RE tiene los complementos de los lenguajes del conjunto RE , es decir, $L \in RE$ si y solo si $L^C \in CO-RE$.

- Se cumple $R \subseteq RE \cap CO-RE$ porque:
 - Por definición, $R \subseteq RE$.
 - Si $L \in R$, entonces $L^C \in R$ (por propiedad), por lo que $L^C \in RE$, por lo que $L \in CO-RE$, por lo que $L \in RE \cap CO-RE$.
- Se cumple $RE \cap CO-RE \subseteq R$ porque:
 - ...
- Por lo tanto tenemos una igualdad: $R = RE \cap CO-RE$.

Las cuatro regiones de la jerarquía de la computabilidad



Las cuatro regiones de la jerarquía,
con grado de dificultad creciente.

1. **Conjunto R .**
 - i. Lenguajes aceptados por MT que siempre paran.
 - ii. Si L está en R , entonces L^C también está en R .
2. **Conjunto $RE - R$.**
 - i. Lenguajes aceptados por MT que no siempre paran.
 - ii. Si L está en RE , entonces L^C está en $CO-RE$.
3. **Conjunto $CO-RE - R$.**
 - i. Lenguajes que no son aceptados por ninguna MT, pero sus complementos sí son aceptados por alguna MT.
 - ii. Si L está en $CO-RE$, entonces L^C está en RE .
4. **Conjunto $\mathcal{L} - (RE \cup CO-RE)$.**
 - i. Lenguajes que no son aceptados por ninguna MT, ni tampoco sus complementos son aceptados por alguna MT.
 - ii. Si L está en $\mathcal{L} - (RE \cup CO-RE)$, entonces L^C también está en $\mathcal{L} - (RE \cup CO-RE)$.

Ejemplos de lenguajes no recursivos

Halting Problem

- Dada una MT M y una cadena de entrada w , ¿ M se detiene a partir de w ?

- Formalmente: $HP = \{(\langle M \rangle, w) \mid M \text{ es una MT que se detiene a partir de } w\}$.
- $\langle M \rangle$ es la codificación de la MT M como una cadena de símbolos de Σ .
- En 1936 Turing demostró que $HP \in RE - R$, es decir, existe una MT que acepta HP , pero no existe una MT que lo decida.
- No existe ninguna MT M' que para **toda** MT M y **toda** cadena de entrada w pueda decidir si M se detiene a partir de w . Sin embargo, para **algunas** MT M y **algunas** cadenas de entrada w , existen MT M' que sí pueden decidir el problema. Lo que sucede es que la solución general es la que no existe.
- Por ejemplo, si M va siempre hacia la derecha, si M se mueve en un espacio acotado de celdas, etc, se puede decidir, usando MT M' "ingeniosas".
- En general siempre se considera el peor caso, es decir, todas las entradas posibles. En dicho caso, el único algoritmo posible para HP es la fuerza bruta (ejecutar M y esperar a que eventualmente responda).

Problema de las ecuaciones diofánticas

- Dada una ecuación algebraica con coeficientes enteros y variables enteras, como por ejemplo $2x^3 + 5y^3 = 6z^3$, ¿la ecuación tiene solución?
- Formalmente: $L = \{\psi \mid \psi \text{ es una ecuación diofántica que tiene solución}\}$
- En 1970 Matiyasevich demostró que $L \in RE - R$, es decir, existe una MT que acepta L , pero no existe una MT que lo decida.

Problema de decisión en la lógica de predicados

- Dada una fórmula Φ en la lógica de predicados, ¿ Φ es un teorema? es decir, existe una demostración de Φ a partir de los axiomas de la lógica de predicados?
- Formalmente: $L = \{\Phi \mid \text{existe una prueba de la fórmula } \Phi \text{ en la LP}\}$
- En 1936 Turing demostró que $L \in RE - R$, es decir, existe una MT que acepta L , pero no existe una MT que lo decida.

Problema de pertenencia al conjunto de Mandelbrot (CM)

- El CM es un conjunto de números complejos del plano definido por la sucesión $z_0 = 0, z_{n+1} = z_n^2 + c$, tal que c está en el conjunto CM si y solo si la sucesión está acotada. El problema dice: dado un número complejo c , ¿ c pertenece al conjunto CM?
- Formalmente: $L = \{c \in \mathbb{C} \mid c \text{ está en el CM}\}$
- En 1989 Blum demostró que $L \in \text{CO-RE} - R$, es decir, no existe una MT que acepte L , pero sí existe una MT que acepta el complemento de L .
- Hay elementos del CM que una MT no puede reconocer. La dificultad está en el contorno del conjunto.
- Existe una MT que acepta todos los números complejos que NO están en el CM.

Problema de decisión en la aritmética

- ¿La fórmula Θ es un enunciado aritmético verdadero?
- Formalmente: $L = \{\Theta \mid \Theta \text{ es un enunciado aritmético verdadero}\}$

- En 1931 Gödel demostró que $L \in \mathcal{L} - (RE \cup CO-RE)$, es decir, no existe una MT que acepte L ni tampoco existe una MT que acepte el complemento de L .
- También se puede expresar que $L \notin RE$ y que $L^C \notin RE$.
- Hay enunciados **verdaderos** que una MT no puede reconocer.
- Hay enunciados **falsos** que una MT no puede reconocer.
- Nota: si excluimos a la multiplicación, el problema se vuelve decidible.

Problema del cubrimiento del plano con polígonos (teselación del plano)

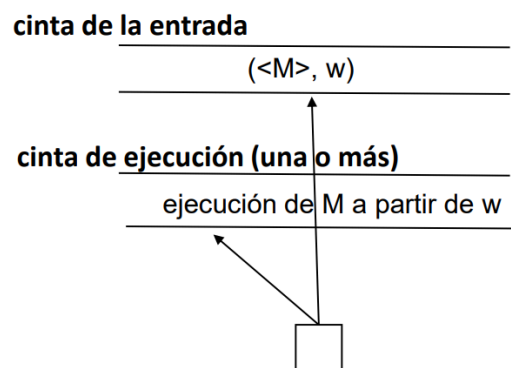
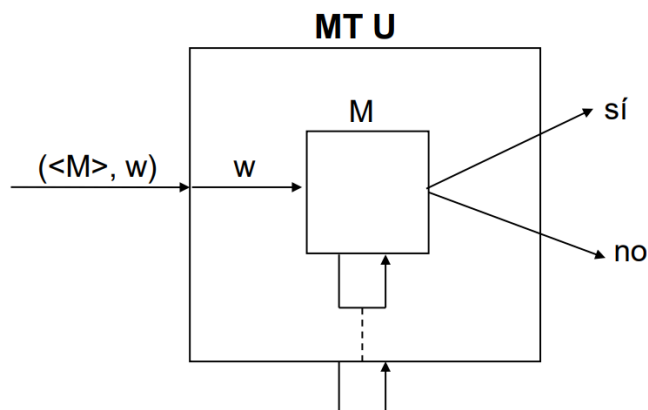
- Dado un conjunto finito C de figuras poligonales conocidas como teselas o mosaicos, ¿C puede cubrir el plano sin dejar huecos ni producir solapamientos?
- Formalmente: $L = \{C \mid C \text{ es un conjunto finito de figuras poligonales que cubren el plano}\}$
- En 1966 Berger demostró que $L \in \mathcal{L} - (RE \cup CO-RE)$, es decir, no existe una MT que acepte L ni tampoco existe una MT que acepte el complemento de L .
- También se puede expresar que $L \notin RE$ y que $L^C \notin RE$.
- Hay conjuntos de polígonos que cubren el plano que una MT no puede reconocer.
- Hay conjuntos de polígonos que no cubren el plano que una MT no puede reconocer.
- Nota: si hay periodicidad, el problema se vuelve decidible.

Clase 3 - 26 de marzo, 2026

Indecibilidad

Máquina de Turing Universal

- Una máquina de Turing Universal (MTU) es una MT capaz de ejecutar cualquier otra MT (noción de programa almacenado, Turing 1936).
- Esquema general:



Con una sola cinta de ejecución le alcanza para ejecutar una MT con cualquier cantidad de cintas

- La MTU recibe una MT M codificada mediante una cadena (por eso se simboliza $\langle M \rangle$) y una cadena de entrada w . Luego ejecuta a la MT M a partir de la cadena de entrada w .

Codificación de una MT y una cadena de entrada

- Hay varias formas de codificar una MT M . La más común es asignar un número natural a cada símbolo del alfabeto Γ de M , a cada estado del conjunto Q de M , y a cada acción posible (moverse a la izquierda, a la derecha o quedarse en el mismo lugar).
- Ejemplo:
 - Sea $M = (Q, \Gamma, \delta, q_0, q_A, q_R)$ con $Q = \{q_0, q_A, q_R\}$ y $\Gamma = \{s, t, B\}$.
 - El código de q_0 es 1.
 - El código de q_A es 2.
 - El código de q_R es 3.
 - El código del símbolo blanco es 1.
 - El código del símbolo s es 2.
 - El código del símbolo t es 3.
 - El código de la acción "moverse a la derecha" es 1.
 - El código de la acción "moverse a la izquierda" es 2.
 - El código de la acción "quedarse en el mismo lugar" es 3.
 - Si la función de transición δ de M es la siguiente:
 - $\delta(q_0, s) = (q_0, t, L)$
 - $\delta(q_0, t) = (q_0, s, L)$
 - $\delta(q_0, B) = (q_A, B, S)$
 - Entonces el código de M , denotado como $\langle M \rangle$, es: $(1, 2, 1, 3, 2), (1, 3, 1, 2, 2), (1, 1, 2, 1, 3)$.
 - Y si la cadena de entrada w es st , entonces tenemos: $(1, 2, 1, 3, 2), (1, 3, 1, 2, 2), (1, 1, 2, 1, 3)\#2, 3$.

Enumeración de las máquinas de Turing

- El hecho de que podemos codificar a cualquier MT como una cadena implica que podemos enumerar a todas las MT que existen, es decir, podemos asignar un número natural a cada MT. - La enumeración más común es el **orden canónico**, donde los códigos de las MT se ordenan primero de menor a mayor longitud, y luego los códigos con longitudes iguales se ordenan lexicográficamente.
- Las primeras cadenas según el orden canónico son:
 - λ (cadena vacía)
 - 0
 - 1
 - 00
 - 01
 - 10
 - 11
 - ...

Probando que $R \subsetneq RE$

- Para probar que $R \subsetneq RE$ hay que probar dos cosas:
 - $R \subseteq RE$.
 - $R \neq RE$.

Tabla con todas las MT y todas las cadenas de entrada

- La siguiente tabla T representa el comportamiento de todas las MT M con respecto a todas las cadenas w :

T	w_0	w_1	w_2	w_3	w_4	$\dots$
M_0	1	0	1	1	1	$\dots$
M_1	1	0	0	1	0	$\dots$
M_2	0	0	1	0	1	$\dots$
M_3	0	1	1	1	1	$\dots$
M_4	0	1	1	1	0	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$

- $T(M_i, w_j) = 1$ si M_i acepta w_j y 0 si M_i rechaza w_j . Los valores en la tabla son puramente de ejemplo.
- La fila 0 representa al lenguaje $L(M_0) = \{w_0, w_2, w_3, w_4, \dots\}$
- La fila 1 representa al lenguaje $L(M_1) = \{w_0, w_3, \dots\}$
- La fila 2 representa al lenguaje $L(M_2) = \{w_2, w_4, \dots\}$
- $\dots$
- Por lo tanto, las filas representan todos los lenguajes posibles aceptados por una MT, es decir, el conjunto RE .

Diagonalización

- La diagonal de la tabla mostrada es $D = \{w_0, w_2, w_3, \dots\}$. Representa al lenguaje $D = \{w_i \mid M_i \text{ acepta } w_i\}$
- La diagonal con los unos y ceros invertidos es $D^C = \{w_1, w_4, \dots\}$. Representa al lenguaje $D^C = \{w_i \mid M_i \text{ rechaza } w_i\}$
- Prueba de que D está en RE : Existe una MT M que acepta D . Dada una cadena de entrada w , M hace lo siguiente:
 - Busca el índice i tal que $w = w_i$ en el orden canónico (genera cadenas en el orden canónico hasta encontrar a w).
 - Busca el código $\langle M_i \rangle$ de la MT M_i en el orden canónico (genera códigos de MT en el orden canónico hasta encontrar a M_i).
 - Ejecuta a M_i a partir de w_i y acepta si y solo si M_i acepta.
 - Por lo tanto se cumple que $D = \{w_i \mid M_i \text{ acepta } w_i\} \in RE$
- Prueba de que D^C no está en RE : La diagonal invertida es diferente de todas las filas de la tabla, porque difiere al menos en un elemento de cada fila, es decir, difiere en el elemento de la diagonal. Como las filas representan a todos los lenguajes que están en RE , entonces D^C es diferente a todos los lenguajes de RE , por lo que $D^C \notin RE$.

- Por lo tanto tenemos que $D \in RE$ y $D^C \notin RE$. Esto implica directamente que $D \in RE - R$, porque si D estuviera en R , entonces D^C también estaría en R (por propiedad de los lenguajes recursivos), lo cual es una contradicción, porque ya se demostró que $D^C \notin RE$ y como $R \subseteq RE$, entonces $D^C \notin R$.
- Por lo tanto, $R \subsetneq RE$.
- Así, se han hallado los primeros dos lenguajes no decidibles: D y D^C . A partir de éstos se pueden encontrar otros lenguajes que también están fuera de R e incluso de RE , usando el famoso método de la **reducción**.
- La diagonalización es muy útil para encontrar los primeros lenguajes en un conjunto. Consiste en encontrar un lenguaje "separador". Por ejemplo, el lenguaje D^C actúa como separador entre los conjuntos RE y $\mathcal{L}$, ya que pertenece a $\mathcal{L}$ pero no a RE .

Ejemplo de diagonalización: los números reales vs los naturales

- Se puede probar por diagonalización que el conjunto de los números reales es **más grande** que el conjunto de los números naturales, lo que implica que los números reales no se pueden enumerar. Simbólicamente: $|\mathbb{R}| > |\mathbb{N}|$.
- Supongamos que lo anterior NO se cumple. Sea por ejemplo la siguiente enumeración de **todos** los reales entre 0 y 1:
 - 0.1287...
 - 0.8550...
 - 0.1380...
 - 0.2751...
 - ...
- Sea el número real 0.1581... que se forma tomando el primer dígito de la primera fila, el segundo dígito de la segunda fila, el tercer dígito de la tercera fila, el cuarto dígito de la cuarta fila, y así sucesivamente. El número 0.1581... es diferente a todos los números de la enumeración, porque difiere al menos en un dígito de cada número de la enumeración, es decir, difiere en el dígito de la diagonal. Por lo tanto, 0.1581... no está en la enumeración, lo cual es una contradicción, porque se asumió que la enumeración contenía a todos los números reales entre 0 y 1. Por lo tanto, $|\mathbb{R}| > |\mathbb{N}|$.

Tamaño de los conjuntos RE y $\mathcal{L}$

- $|\mathbb{N}| = |RE|$ porque las MT se pueden enumerar, y entonces los lenguajes de RE también se pueden enumerar.
- $|\mathbb{R}| = |\mathcal{L}|$ porque:
 - $|\mathbb{R}| = |P(\mathbb{N})|$ porque $|\mathbb{N}| = |\Sigma^*|$ porque las cadenas de Σ^* se pueden enumerar. Así, $|P(\mathbb{N})| = |P(\Sigma^*)|$. $P(\Sigma^*) = \mathcal{L}$ porque $\mathcal{L}$ es el conjunto de partes de Σ^* . Así, finalmente, $|P(\Sigma^*)| = |\mathcal{L}|$.
 - Resumiendo: $|\mathbb{R}| = |P(\mathbb{N})| = |P(\Sigma^*)| = |\mathcal{L}|$
- Como $|\mathbb{N}| = |RE|$ y $|\mathbb{R}| = |\mathcal{L}|$, entonces $\mathcal{L}$ es mucho más grande que RE , lo que implica que hay muchos más problemas no computables que problemas computables, es decir, la mayoría de los problemas son no computables. A su vez, como se sabe que $R \subsetneq RE$, entonces también hay muchos más problemas computables no decidibles que problemas computables decidibles, es decir, la mayoría de los problemas computables son no decidibles. El conjunto de problemas que más nos interesa (R) es el más pequeño de todos.

Dificultad del Halting Problem

- HP es uno de los lenguajes más difíciles de RE .
- Si HP fuera decidable, entonces todo lenguaje de RE sería decidable también:

- Sea M_{HP} una MT que decide HP y M una MT que reconoce algún lenguaje L de RE . La siguiente MT M' decide L :
 - M' primero decide si M para a partir de w .
 - Si M no para a partir de w , entonces M' rechaza.
 - Si M para a partir de w , entonces M' ejecuta a M a partir de w y responde igual que M .
- HP está en la frontera de RE , lo más lejos posible de R . Se dice que es RE — *completo*, identifica el grado de dificultad de RE , representa el problema general de la indecidibilidad.
- Si HP fuera decidable muchos enunciados matemáticos se probarían fácilmente, como por ejemplo la conjetura de Goldbach o el último teorema de Fermat (que sí fue resuelto eventualmente).

Computabilidad vs. tamaño de un lenguaje

- La computabilidad de un lenguaje tiene mucho más que ver con su definición, su contorno, que con su tamaño.
- $HP \subseteq \Sigma^*$ y sin embargo HP es más difícil que Σ^* , porque HP no es decidable mientras que Σ^* sí.
- $HP^C \subseteq \Sigma^*$ y sin embargo HP^C es más difícil que Σ^* , porque HP^C no es recursivamente enumerable mientras que Σ^* es decidable.
- El conjunto de Mandelbrot, perteneciente a $CO-RE - R$, ilustra muy bien la relación entre computabilidad y contorno.

La demostración de Turing de que HP no es decidable

- Se supone que HP es decidable y se llega a una contradicción.
- Sea M_{HP} una MT que decide HP . Utilizando M_{HP} se construye una MT C que hace lo siguiente:
 - i. Si M_{HP} dice que M se detiene a partir de w , entonces C loopea.
 - ii. Si M_{HP} dice que M no se detiene a partir de w , entonces C acepta.
 - iii. Es decir, C le lleva la contra a M_{HP} .
- Si la entrada de C es su propio código $\langle C \rangle$, entonces:
 - i. Si M_{HP} dice que C se detiene desde $\langle C \rangle$, entonces C loopea desde $\langle C \rangle$, absurdo.
 - ii. Si M_{HP} dice que C no se detiene desde $\langle C \rangle$, entonces C se detiene desde $\langle C \rangle$, absurdo.
- Así, C no puede existir, y como se construyó a partir de M_{HP} , entonces M_{HP} tampoco puede existir, por lo que HP no es decidable.

Autorreferencia

- La demostración de Turing de que HP no es decidable se basa en la autorreferencia, es decir, en el hecho de que una MT pueda recibir su propio código como entrada.
- La autorreferencia es muy útil para probar algunos enunciados.
- Se caracteriza por encontrar paradojas, es decir, enunciados que se contradicen a sí mismos.
- Ejemplos:
 - **Paradoja del mentiroso:** Si Juan dice "estoy mintiendo", ¿está diciendo la verdad o está mintiendo? Si está diciendo la verdad, entonces está mintiendo, lo cual es una contradicción. Si está mintiendo, entonces está diciendo la verdad, lo cual también es una contradicción.
 - **Paradoja de Russell:** Sea U el conjunto de todos los conjuntos que no son elementos de sí mismos. ¿ U es un elemento de sí mismo? Si U es un elemento de sí mismo, entonces no es un conjunto que no es elemento de sí

mismo, lo cual es una contradicción. Si U no es un elemento de sí mismo, entonces es un conjunto que no es elemento de sí mismo, lo cual también es una contradicción.

- **Paradoja del barbero:** En un pueblo hay un barbero que afeita a todos los hombres que no se afeitan a sí mismos. ¿El barbero se afeita a sí mismo? Si el barbero se afeita a sí mismo, entonces no se afeita a sí mismo, lo cual es una contradicción. Si el barbero no se afeita a sí mismo, entonces se afeita a sí mismo, lo cual también es una contradicción.

Clase 4 - 31 de marzo, 2026

Resumen de lenguajes vistos hasta ahora

- $L = \{a^n b^n \mid n \geq 1\}$ pertenece a R , y se demostró por construcción.
- HP : pertenece a $RE - R$, y se demostró por construcción.
- HP^C : pertenece a $CO-RE - R$, y se demostró por diagonalización.
- D : pertenece a $RE - R$, y se demostró por construcción.
- D^C : pertenece a $CO-RE - R$, y se demostró por diagonalización.
- **Para probar pertenencia a R o a RE se construyen MTs.**
- **Para probar no pertenencia a R o a RE se usa diagonalización.**
- Otra forma de hacer lo anterior es usando el método de la reducción.

Reducción

Definición

- Dados dos lenguajes L_1 y L_2 , una reducción de L_1 a L_2 es una función total computable $f : \Sigma^* \rightarrow \Sigma^*$ definida para todas las cadenas de Σ^* y computable por una MT M_f , tal que para toda cadena w :
 - Si $w \in L_1$, entonces $f(w) \in L_2$.
 - Si $w \notin L_1$, entonces $f(w) \notin L_2$.
- El objetivo es reducir o relacionar el lenguaje L_1 al lenguaje L_2 con la idea de construir una MT M_1 que acepte L_1 utilizando una MT M_2 ya conocida que acepta L_2 , en lugar de construir a M_1 desde cero.
- Las reducciones se asemejan a las invocaciones a subrutinas en la programación.
- Si L_1 se reduce a L_2 se denota $L_1 \leq L_2$.
- **Intuitivamente, si $L_1 \leq L_2$, entonces L_2 es tan o más difícil que L_1 .**

Propiedad sobre los lenguajes recursivos

- Sean L_1 y L_2 lenguajes tales que $L_1 \leq L_2$:
 - Si $L_2 \in R$, entonces $L_1 \in R$.
 - Equivalentemente: Si $L_1 \notin R$, entonces $L_2 \notin R$.

- Encontrando una reducción de un lenguaje L_1 que no está en R a un lenguaje L_2 , se prueba que L_2 tampoco está en R .

Propiedad sobre los lenguajes recursivamente enumerables

- Sean L_1 y L_2 lenguajes tales que $L_1 \leq L_2$:
 - Si $L_2 \in RE$, entonces $L_1 \in RE$.
 - Equivalentemente: Si $L_1 \notin RE$, entonces $L_2 \notin RE$.
- Encontrando una reducción de un lenguaje L_1 que no está en RE a un lenguaje L_2 , se prueba que L_2 tampoco está en RE .

Ejemplo 1: Reducción de HP a L_U

- $HP = \{(\langle M \rangle, w) \mid M \text{ es una MT que se detiene a partir de } w\}$
- $L_U = \{(\langle M \rangle, w) \mid M \text{ es una MT que acepta } w\}$
- El objetivo de esta reducción es probar que $L_U \notin R$, apoyándonos en dos cosas:
 - i. Sabemos que $HP \notin R$
 - ii. La propiedad de los lenguajes recursivos que dice que si $L_1 \leq L_2$ y $L_1 \notin R$, entonces $L_2 \notin R$.
- Algo clave a notar es la relación que hay entre los dos lenguajes. Los pares $(\langle M \rangle, w)$ que pertenecen a HP son MTs que se detienen tanto en q_A como en q_R , mientras que los pares $(\langle M \rangle, w)$ que pertenecen a L_U son MTs que se detienen en q_A pero no en q_R . Es decir, $L_U \subset HP$.
- Reducción:
 - Se define $f(\langle M \rangle, w) = (\langle M' \rangle, w)$ donde M' es una MT igual a M pero todas las transiciones a q_R se cambian por transiciones a q_A .
 - Luego tenemos varios casos:
 - Si $(\langle M \rangle, w) \in HP$, entonces M se detiene a partir de w , lo que implica que M' también se detiene a partir de w , en particular aceptando, lo que implica que $(\langle M' \rangle, w) \in L_U$.
 - Si $(\langle M \rangle, w) \notin HP$, entonces M no se detiene a partir de w , lo que implica que M' tampoco se detiene a partir de w , porque haber cambiado todos los q_R por q_A no evita que M' loopee, lo que implica que $(\langle M' \rangle, w) \notin L_U$.
 - Si $(\langle M \rangle, w)$ es basura, es decir contiene una MT malformada, no pertenece a HP , y luego de la modificación tampoco pertenece a L_U .
 - La función f es **computable** porque transformar la descripción de M reemplazando estados es un procedimiento mecánico que siempre termina.
 - Por lo tanto se cumple que f es una reducción de HP a L_U , es decir, $HP \leq L_U$.
 - Luego, como $HP \notin R$ y $HP \leq L_U$, entonces $L_U \notin R$.

Ejemplo 2: Reducción de L_{Σ^*} a L_{EQ}

- $L_{\Sigma^*} = \{\langle M \rangle \mid L(M) = \Sigma^*\}$
- $L_{EQ} = \{(\langle M_1 \rangle, \langle M_2 \rangle) \mid L(M_1) = L(M_2)\}$
- El objetivo de esta reducción es probar que $L_{EQ} \notin RE$, apoyándonos en dos cosas:
 - i. Sabemos que $L_{\Sigma^*} \notin RE$

- ii. La propiedad de los lenguajes recursivos que dice que si $L_1 \leq L_2$ y $L_1 \notin RE$, entonces $L_2 \notin RE$.
- Reducción:
 - Se define $f(\langle M \rangle) = (\langle M \rangle, \langle T \rangle)$ donde M es una MT igual a M y T es una MT que acepta Σ^* .
 - Luego tenemos varios casos:
 - Si $(\langle M \rangle) \in L_{\Sigma^*}$, entonces M acepta Σ^* , lo que implica que M y T aceptan el mismo lenguaje, lo que implica que $(\langle M \rangle, \langle T \rangle) \in L_{EQ}$.
 - Si $(\langle M \rangle) \notin L_{\Sigma^*}$, entonces M no acepta Σ^* , lo que implica que M y T no aceptan el mismo lenguaje, lo que implica que $(\langle M \rangle, \langle T \rangle) \notin L_{EQ}$.
 - Si $(\langle M \rangle)$ es basura, es decir contiene una MT malformada, no pertenece a L_{Σ^*} , y luego de la modificación tampoco pertenece a L_{EQ} .
 - La función f es **computable** porque simplemente se le concatena a la descripción de M la descripción de T , lo cual es un procedimiento mecánico que siempre termina.
 - Por lo tanto se cumple que f es una reducción de L_{Σ^*} a L_{EQ} , es decir, $L_{\Sigma^*} \leq L_{EQ}$.
 - Luego, como $L_{\Sigma^*} \notin RE$ y $L_{\Sigma^*} \leq L_{EQ}$, entonces $L_{EQ} \notin RE$.

Propiedades de las reducciones

- **Reflexividad:** Para todo lenguaje L se cumple $L \leq L$. Se prueba trivialmente usando la función identidad $f(w) = w$.
- **Transitividad:** Si $L_1 \leq L_2$ y $L_2 \leq L_3$, entonces $L_1 \leq L_3$. Se prueba usando la composición de las dos reducciones, haciendo uso del concepto de composición de funciones.
- No se cumple la **simetría**, es decir, $L_1 \leq L_2$ no implica $L_2 \leq L_1$.
- $L_1 \leq L_2$ si y solo si $L_1^C \leq L_2^C$. La reducción es la misma.

Probando que HP está entre los lenguajes más difíciles de RE

- Se puede probar que todo lenguaje $L \in RE$ cumple $L \leq HP$, es decir, que HP es tan o más difícil que cualquier lenguaje de RE .
- Decidiendo HP se decide cualquier lenguaje L_i . Así, si HP fuera decidible, que no lo es, se cumpliría $R = RE$.
- Se prueba:
 - Dada M , una MT que acepta L .
 - Se construye M' que es igual a M pero cambiando todas las transiciones que iban a q_R por **loops**.
 - Si $w \in L$, entonces M acepta w , lo que implica que M' se detiene a partir de w .
 - Si $w \notin L$, entonces M rechaza w , lo que implica que M' no se detiene a partir de w .

Autómatas finitos

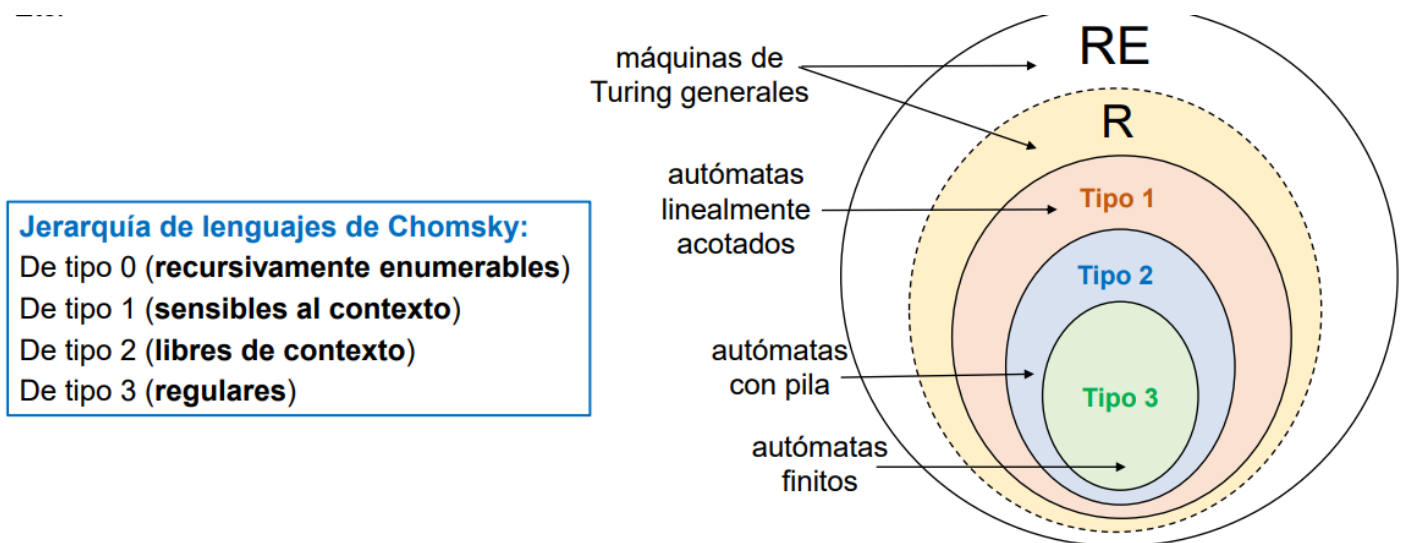
- Se tiene una cinta de solo lectura.
- Solo se puede mover hacia la derecha.
- El conjunto F contiene a los estados finales.
- Cuando el autómata llega al símbolo blanco (B) se detiene y acepta si y solo si el estado alcanzado es un estado final, es decir, si $q \in F$.
- Este tipo de autómatas son muy usados en la práctica, por ejemplo, para el análisis sintáctico de los compiladores, o para las inspecciones de código en QA.

- Se suelen representar mediante diagramas de transición de estados.
- Una de las grandes utilidades de estos autómatas es que **siempre se detienen**, es decir, todo AF pertenece a R .
- Aceptan un tipo limitado de cadenas, ya que no tienen memoria. No pueden calcular cosas en general.
- Los lenguajes aceptados por AF se llaman **lenguajes regulares** o de tipo 3.

Autómatas con pila

- Se tiene una primera cinta de solo lectura.
- Se tiene una segunda cinta de lectura/escritura que se comporta como una pila (stack).
- En un mismo paso se pueden procesar las dos cintas.
- En la primera cinta siempre se va hacia la derecha.
- Cuando el autómata llega al símbolo blanco (B) en la primera cinta, se detiene y acepta si y solo si la pila está vacía.
- Algunos problemas típicos que este tipo de autómatas resuelven es el análisis sintáctico de los lenguajes de programación, o la evaluación de expresiones en la ejecución de programas.
- Al igual que los AF, los AP siempre se detienen, es decir, todo AP pertenece a R .
- Los lenguajes aceptados por AP se llaman **lenguajes libres de contexto** o de tipo 2.

Jerarquía de Chomsky



Enumeración de los lenguajes recursivamente enumerables

- Todo lenguaje $L \in RE$ se puede enumerar. Sea M_1 una MT que acepta L . Se construye una MT M_2 que genera L :
 - $n = 1$
 - Generar todas las cadenas de longitud a lo sumo n en el orden canónico.
 - Por cada cadena generada, ejecutar a lo sumo n pasos de la MT M_1 . Si M_1 acepta, imprimir la cadena.
 - $n = n + 1$ y volver al paso 2.
- Todo lenguaje $L \in R$ se puede enumerar en el orden canónico. Sea M_1 una MT que decide L . Se construye una MT M_2 que genera L en el orden canónico:
 - Generar la primer cadena en el orden canónico.

- ii. Ejecutar M_1 a partir de la cadena generada. Si M_1 acepta, imprimir la cadena.
- iii. Generar la siguiente cadena en el orden canónico y volver al paso 2.
- Por último, dada una MT que enumera un lenguaje, se puede construir una MT que lo acepta.

Clase 5 - 7 de abril, 2026

Introducción

- Dentro de la clase R se pueden identificar dos tipos de problemas, tratables e intratables.
- Los problemas intratables no poseen algoritmos eficientes para resolverlos.
- Los problemas tratables sí poseen algoritmos eficientes para resolverlos.
- Las dos métricas de complejidad computacional más usadas son:
 - Tiempo: cantidad de pasos que ejecuta la MT para resolver el problema.
 - Espacio: cantidad de celdas de la cinta que utiliza la MT para resolver el problema.
- En la práctica los problemas indecidibles se "igualan" a los problemas intratables por más que estos últimos sí sean decidibles, ya que tardan demasiado tiempo en ser resueltos como para ser útiles en el mundo real.
- Ejemplos de problemas tratables:
 - Operaciones aritméticas básicas.
 - Búsqueda de un camino en un grafo.
 - Ordenamiento de un arreglo.
 - Máximo común divisor.
 - Búsqueda del máximo flujo en una red.
 - Multiplicación de matrices.

Complejidad temporal

Concepto

- Una MT M tarda más (hace más pasos) a medida que sus cadenas de entrada w son más grandes.
- Es por esto que el tiempo de ejecución de M no se mide en términos absolutos sino con funciones temporales $T(n)$ que se definen en términos del tamaño de la cadena de entrada $n = |w|$.

Ejemplos de funciones temporales

- Funciones polinomiales o $poly(n)$ con n como **factor**:
 - $T(n) = 5n + 8$
 - $T(n) = 3n^2$
 - $T(n) = 3n^3 + n^2 + 25n$
- Funciones exponenciales o $exp(n)$ con n como **exponente**:

- $T(n) = 20^{\log_2 n}$
- $T(n) = 12^n + 10n + 5$
- $T(n) = 6^{n^5}$

Definición de orden O

- Una función $T_1(n)$ es del orden de una función $T_2(n)$, denotado como $T_1(n) = O(T_2(n))$, si y solo si para todo $n \geq n_0$, existe una constante $c > 0$ tal que $T_1(n) \leq c \cdot T_2(n)$.
- Ejemplos:
 - $5n^3 + 8n + 25 = O(n^3)$
 - $n^2 = O(n^3)$
 - $n^3 = O(2^n)$
 - $n \cdot \log_2 n = O(n^2)$

$TIME(T(n))$

- Una MT M tarda tiempo $T(n)$ si y solo si para toda cadena de entrada w con $n = |w|$, M hace a lo sumo $T(n)$ pasos.
- Un lenguaje $L \in TIME(T(n))$ si y solo si existe una MT M que lo decide en tiempo $O(T(n))$.
- Se considera el tiempo máximo (procesar cualquier cadena w consume a lo sumo $T(|w|)$ pasos).
- Otros criterios son el tiempo promedio y el tiempo mínimo, en general muy difíciles de calcular.

Tesis Fuerte de Church-Turing

- Si L es decidible en tiempo $poly(n)$ por un modelo computacional físicamente realizable entonces también es decidible en tiempo $poly(n)$ por una MT, al menos hasta que las máquinas cuánticas sean una realidad.

Codificación de las cadenas

- Todo símbolo se codifica con un número.
- Se utiliza cualquier codificación que NO SEA la **unaria**, ya que codificar números grandes en unario no es físicamente realizable y además genera inconsistencias.
- Para uniformar usaremos siempre la codificación binaria, es decir, cada símbolo se codifica con una cadena de bits.
- El tamaño n de un número X en binario es $O(\log_2 X)$. Por ejemplo:
 - $X = 29$ se codifica como 11101. $n = 5 \approx \log_2 29$.

La clase P

- Todo lenguaje que es aceptado por una MT en tiempo polinomial pertenece a la clase P .
- Coloquialmente, P es la clase de los problemas tratables, es decir, aquellos que poseen algoritmos eficientes para ser resueltos.

- Si un problema está en P podemos asegurar que sus soluciones se **encuentran** eficientemente.

La clase NP

- Todo lenguaje L de NP cuenta con una MT M que en tiempo polinomial puede verificar si $w \in L$ con la ayuda de otra cadena de entrada c llamada **certificado**.
- Si un problema está en NP podemos asegurar que sus soluciones se **verifican** eficientemente.

P vs NP

- Este problema fue planteado hace más de 50 años y es uno de los siete problemas del milenio, es decir, uno de los problemas más importantes de la matemática sin resolver. Quien lo resuelva se llevará un premio de un millón de dólares.
- No parece posible resolverlo usando las matemáticas actuales.
- Miles de lenguajes útiles e interesantes se encuentran en $NP - P$: lenguajes relacionados con problemas de la lógica, grafos, aritmética, teoría de conjuntos, álgebra, combinatoria, redes, etc.
- Pareciera que la única forma de decidir los lenguajes de $NP - P$ hoy por hoy es por medio de la **fuerza bruta**, es decir, probando todas las posibles soluciones hasta encontrar una que funcione, lo cual no es eficiente.
- Por el contrario, los algoritmos que deciden los lenguajes de P son **analíticos**, parten del conocimiento profundo del problema y fueron elaborados a partir del ingenio, creatividad, experiencia, etc.
- Se sabe que $P \subseteq NP$, pero no se sabe si $P = NP$ o $P \neq NP$.
- Intuitivamente se suele asumir que $P \neq NP$ ya que encontrar una solución suele ser más difícil que verificarla, pero no hay una demostración formal.
- La mayoría de los investigadores opina que $P \neq NP$. Una minoría opina lo contrario, argumentando que queda mucho por aprender de la algoritmia, cosa que es razonable ya que a lo largo de los años han habido importantes descubrimientos en el campo, como la primalidad en P , mejora en los tiempos de multiplicación de matrices, camino en grafos no dirigidos en espacio logarítmico, etc.
- En la práctica se asume sin demostración que $P \neq NP$ y por eso a los lenguajes de interés de $NP - P$ se los procesa con remediaciones específicas como cadenas de determinada longitud/forma, aproximaciones polinomiales, etc.

Complementos

- P es cerrada bajo complementos, es decir, si $L \in P$, entonces $L^C \in P$. Por lo tanto $P = \text{CO-}P$.
- NP se **cree** que no es cerrada bajo complementos, es decir, si $L \in NP$, entonces L^C no necesariamente pertenece a NP . Por lo tanto la conjetura es que $NP \neq \text{CO-}NP$.
- Esta diferencia refuerza la intuición de que $P \neq NP$.

Clase 6 - 14 de abril, 2026

Reducción polinomial

- Una reducción polinomial de un lenguaje L_1 a un lenguaje L_2 es una reducción de L_1 a L_2 que trabaja en tiempo polinomial. Se denota $L_1 \leq_p L_2$.
- **Teorema:** Si $L_1 \leq_p L_2$ y $L_2 \in P$, entonces $L_1 \in P$.

Clase NPC

Concepto

- Un lenguaje L es NPC (NP-completo) si y solo si:
 - i. $L \in NP$
 - ii. Para todo lenguaje $L_i \in NP$, se cumple que $L_i \leq_p L$. Se dice que L es NPH , o NP-difícil.
- Una forma de reforzar la sospecha de que un lenguaje de NP no está en P es probando que es NPC .
- Si se prueba que un lenguaje NPC está en P , entonces se prueba que $P = NP$.
- Si se cumple $P \neq NP$, entonces ningún lenguaje NPC está en P .

El lenguaje SAT

- SAT es el lenguaje de las fórmulas booleanas satisfactibles, es decir, aquellas fórmulas que pueden ser evaluadas como verdaderas para alguna asignación de valores a sus variables.
- Formalmente, $SAT = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores y satisfactible}\}$.
- Este lenguaje está en NPC , lo que implica que todos los lenguajes de NP se reducen polinomialmente a SAT .
- La demostración de que SAT es NPC es similar a la que usó Turing en 1936 para probar que la lógica de predicados no es decidible: Dado cualquier $L \in NP$ la MT M_f asigna a toda cadena w en tiempo polinomial una fórmula booleana ϕ tal que:
 - Si $w \in L$, entonces ϕ es satisfactible, es decir, $\phi \in SAT$.
 - Si $w \notin L$, entonces ϕ no es satisfactible, es decir, $\phi \notin SAT$.

Cómo encontrar más lenguajes NPC

1. Probar que $L \in NP$.
2. Construir una reducción polinomial de SAT a L , es decir, $SAT \leq_p L$.

Propiedades importantes de los problemas NPC

- Hay varias heurísticas para poblar la clase NPC con reducciones polinomiales. Un compendio insuperable es el libro de Garey y Johnson de 1979.

- Levin llamó a los problemas ***NPC* problemas universales**. La idea es que los miles de problemas de *NPC* son en realidad pocos problemas pero que se repiten y se expresan de distintas formas (por medio de grafos, fórmulas booleanas, números, conjuntos, etc).
- Concepto de **p-isomorfismo**:
 - Todo par de lenguajes L_1 y L_2 de *NPC* son p-isomorfos, lo que significa que la función f y la función inversa f^{-1} son ambas reducciones polinomiales. Es decir, f es una reducción polinomial de L_1 a L_2 y f^{-1} es una reducción polinomial de L_2 a L_1 .
 - A partir de esto se concluye que si se puede probar que todos los lenguajes *NPC* son p-isomorfos, entonces se cumple $P \neq NP$.
- Todos los lenguajes *NPC* son **densos**, es decir que tienen muchas cadenas. Formalmente, para todo n tienen $exp(n)$ cadenas de longitud a lo sumo n .
 - Se concluye que si existe un lenguaje *NPC* que no es denso (se le dice disperso), entonces se cumple $P = NP$.

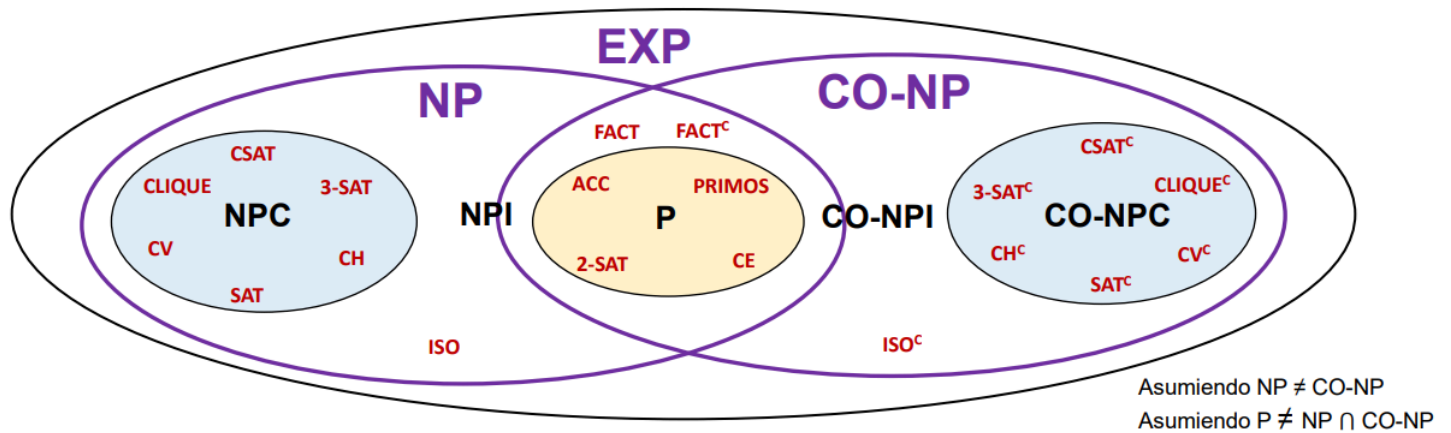
Computabilidad vs complejidad

- A diferencia de la computabilidad, en la complejidad computacional el tamaño de un lenguaje sí tiene mucho que ver con su dificultad.
- Los lenguajes completos de una clase identifican la dificultad de toda la clase.
 - En la computabilidad todo lenguaje de *RE* se reduce a *HP*, lo que implica que *HP* es tan o más difícil que todos los lenguajes de *RE*. ***HP* identifica la dificultad de la clase *RE***.
 - En la complejidad temporal todo lenguaje de *NP* se reduce polinomialmente a *SAT*, lo que implica que *SAT* es tan o más difícil que todos los lenguajes de *NP*. ***SAT* identifica la dificultad de la clase *NP***.

Clase *NPI*

- Asumiendo $P \neq NP$, entre las clases P y *NPC* se encuentra la clase *NPI* (NP-intermedio), llamada así porque identifica a los lenguajes de *NP* de dificultad intermedia.
- En esta clase se ubican los lenguajes que no cuentan con una MT de tiempo polinomial que los decida y además no cuentan con una reducción polinomial de un lenguaje *NPC* hacia ellos.
- Ejemplo:
 - Problema de los grafos isomorfos "ISO".
 - ISO está en *NP*, pero se asume que no está en P ni en *NPC*, por lo que caería en *NPI*.

Jerarquía de complejidad temporal



- Se cumple que $NP \neq CO-NP$ implica que $P \neq NP$.
- Se cumple que NPC y $NP \cap CO-NP$ son disjuntos, es decir, no hay ningún lenguaje que sea a la vez NPC y $NP \cap CO-NP$.
- Regiones ordenadas por su dificultad, de la más fácil a la más difícil:
 - P
 - $(NP \cap CO-NP) - P$
 - $NPI - (NP \cap CO-NP)$
 - NPC
 - $CO-NPI - (NP \cap CO-NP)$
 - $CO-NPC$

Clase 7 - 21 de abril, 2026

Complejidad espacial

Concepto

- Una MT M ocupa espacio $S(n)$ si y solo si al ejecutarse desde toda entrada w con $n = |w|$, M ocupa a lo sumo $S(n)$ celdas en cualquier cinta que no sea la cinta de entrada.
- La cinta de entrada es de sólo lectura y no se considera en la medición del espacio. Esto permite un espacio menor que lineal (menor que $O(n)$) porque la cadena de entrada mide n .
- En el caso más general con una cinta de salida, ésta tampoco se considera en la medición del espacio.

Clase $SPACE(S(n))$

- $L \in SPACE(S(n))$ si y solo si existe una MT M que decide L en espacio $O(S(n))$.

- Una MT M de tiempo $T(n)$ ocupa a lo sumo $T(n)$ celdas. Esto se debe a que la MT recorre no más de $T(n)$ celdas en una dirección.
- En cambio, una MT M que ocupa espacio $S(n)$ puede tardar mucho más que tiempo $S(n)$, ya que puede looppear en las mismas celdas durante mucho tiempo.

Clase $PSPACE$

- $L \in PSPACE$ si y solo si existe una MT M que decide L en espacio polinomial, es decir, en espacio $O(n^k)$ para alguna constante $k > 0$.

Clase $LOGSPACE$

- $L \in LOGSPACE$ si y solo si existe una MT M que decide L en espacio logarítmico, es decir, en espacio $O(\log_2 n)$.

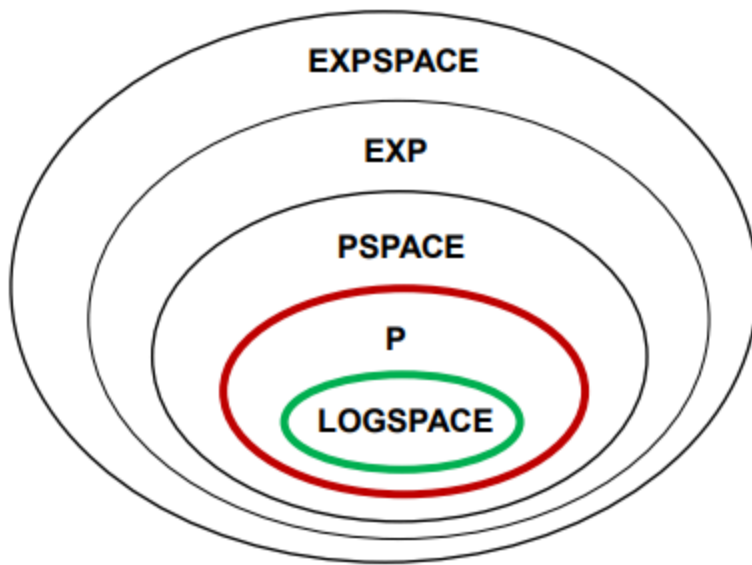
Clase $EXPSPACE$

- $L \in EXPSPACE$ si y solo si existe una MT M que decide L en espacio exponencial, es decir, en espacio $O(2^{n^k})$ para alguna constante $k > 0$.

Relación entre $LOGSPACE$ y P

- Espacio $S(n)$ implica tiempo $c^{S(n)}$ para alguna constante $c > 0$.
- Si $S(n) = \log_2 n$, entonces tenemos tiempo $c^{\log_2 n} = n^{\log_2 c} = n^k$ (definición de polinomio)
- Por lo tanto $LOGSPACE \subseteq P$, es decir, si una MT decide un lenguaje en **espacio** logarítmico, entonces también lo decide en **tiempo** polinomial.
- Dicho de otra forma, los lenguajes de $LOGSPACE$ son tratables.

Versión 1 de la jerarquía espacio-temporal



Se conjetura que $\text{LOGSPACE} \subset P$

Se conjetura que $P \subset \text{PSPACE}$

Se prueba que $\text{LOGSPACE} \subset \text{PSPACE}$

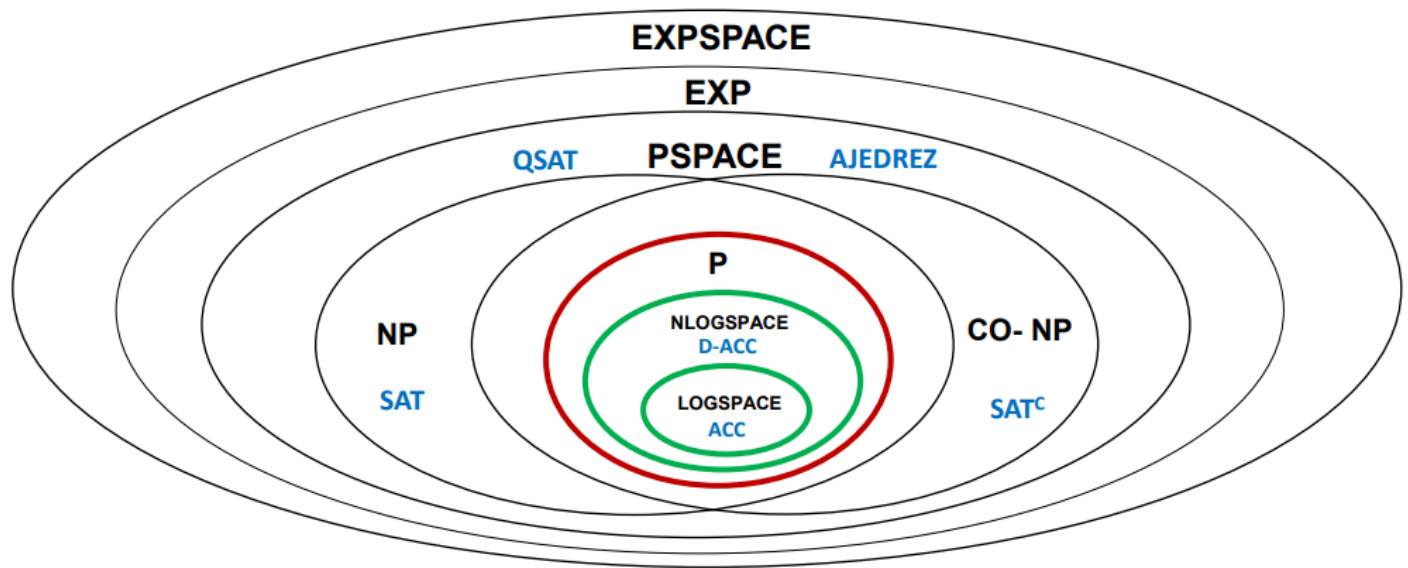
Se conjetura que $\text{PSPACE} \subset \text{EXP}$

Se prueba que $\text{EXP} \subset \text{EXPSPACE}$

Clase $N\text{LOGSPACE}$

- $L \in N\text{LOGSPACE}$ si y solo si existe una MT M que lo verifica en espacio logarítmico, es decir, en espacio $O(\log_2 |w|)$ con la ayuda de un certificado sucinto c , es decir, $|c| \leq \text{poly}(|w|)$. El certificado está en una cinta auxiliar de sólo lectura y se procesa símbolo a símbolo, yendo solo hacia la derecha.

Versión 2 de la jerarquía espacio-temporal



Se sospecha

$\text{LOGSPACE} \subset \text{NLOGSPACE} \subset \text{P} \subset \text{PSPACE} \subset \text{EXP}$

Se cumple

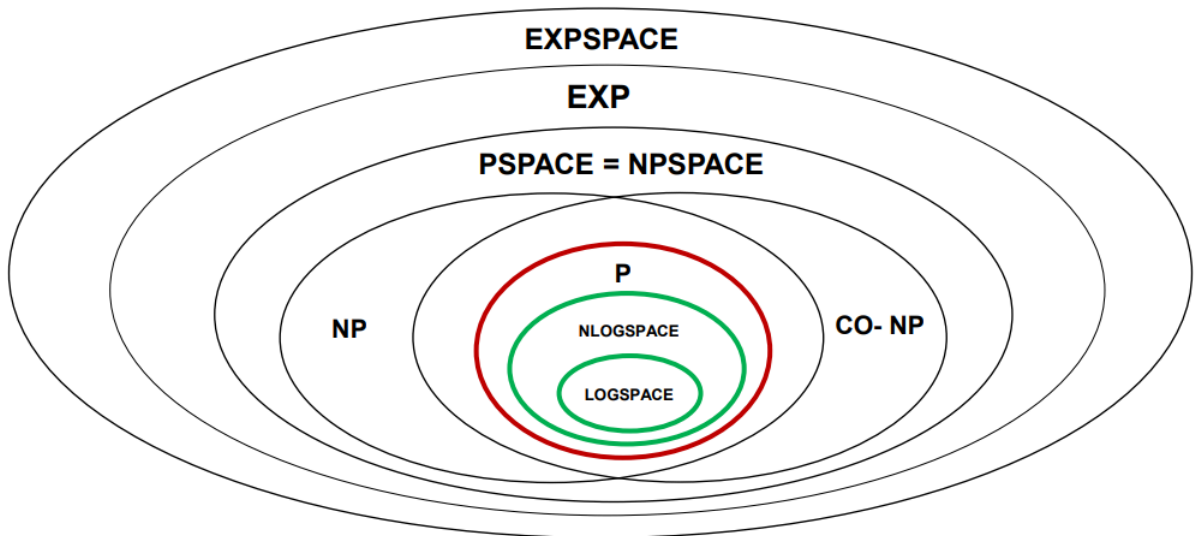
$\text{LOGSPACE} \subset \text{PSPACE}$

$\text{P} \subset \text{EXP}$

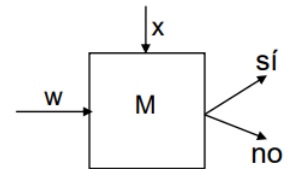
Complejidad

- Un lenguaje es PSPACE-completo si y solo si todo lenguaje L de PSPACE se reduce a él en **tiempo polinomial**.
 - $\text{QSAT} = \{\phi \mid \phi \text{ es una fórmula booleana con cuantificadores y satisfactible}\}$.
 - QSAT es PSPACE – completo.
- Un lenguaje es NLOGSPACE-completo si y solo si todo lenguaje L de NLOGSPACE se reduce a él en **espacio logarítmico**.
 - $\text{D-ACC} = \{G \mid G \text{ es un grafo dirigido con un camino del primero al último vértice}\}$.
 - D-ACC es NLOGSPACE – completo.

Versión 3 de la jerarquía espacio-temporal



- Un lenguaje L pertenece a **NPSPACE** si existe una MT M que lo **verifica** en espacio $O(\text{poly}(n))$, con la ayuda de un **certificado** x que se usa de una manera apropiada.
- Se cumple **PSPACE = NPSPACE**, o sea que en espacio polinomial, decidir y verificar tienen el mismo costo computacional.



Clase 8 - 19 de mayo, 2026

Verificación axiomática de programas

Introducción

- La **verificación** es una actividad formal. La **validación** es una actividad informal, fundamentalmente el testing.
- Dijkstra decía "El testing asegura la presencia de errores pero no su ausencia". Es decir, el testing es útil para encontrar errores pero no puede nunca asegurar que un programa esté totalmente libre de errores.
- Hoare decía "Todas las propiedades de un programa pueden probarse en principio a partir de su propio texto por medio del puro razonamiento deductivo"
- Dijkstra fue uno de los pioneros de lo que hoy se conoce como TDD (Test Driven Development), ya que decía "Pensar cómo sería la prueba de un programa, y luego construirlo siguiendo la estructura de la prueba, para así construirlo y probarlo al mismo tiempo y obtener un programa que es correcto por construcción".

Especificación vs Programa

- La **especificación** tiene mucho que ver con la lógica de predicados, la lógica temporal, etc.
- El **programa** es el código, que puede ser imperativo, funcional, etc.

Formas de verificación

- **Semántica:** se usa la semántica de las instrucciones del lenguaje. Se complica mucho cuando los programas son complejos (sobre todo los concurrentes).
- **Sintáctica:** se usan axiomas y reglas de un método deductivo. Se le llama Lógica de Hoare, y es el método más usado en la práctica.

Lógica de Hoare

- **Terna de hoare:** $\{p\}C\{q\}$
 - p es la precondition.
 - C es el programa o fragmento de código a verificar.
 - q es la postcondición.
- **Precondición:** p
 - Es un predicado que se asume verdadero antes de ejecutar el programa.
- **Postcondición:** q
 - Es un predicado que se asume verdadero después de ejecutar el programa.
- **Especificación:** (p, q)
 - Es el par de precondition y postcondición.
- **Estado:** σ
 - Es una función que asigna a toda variable un valor. Por ejemplo, $\sigma(x) = 1, \sigma(y) = 2$, etc.
- **Satisfacción:**
 - Un estado σ satisface a un predicado p , denotado $\sigma \models p$, si y solo si p es verdadero cuando se evalúa con los valores asignados por σ a sus variables.
 - Por ejemplo, si p es $x < y$ y $\sigma(x) = 1, \sigma(y) = 2$, entonces $\sigma \models p$ porque $1 < 2$ es verdadero.
- **Correctitud:**
 - Un programa S es correcto con respecto a una especificación (p, q) , denotado $\{p\}S\{q\}$, si y solo si para todo estado σ , si $\sigma \models p$ entonces S ejecutado a partir de σ se detiene en un estado σ' tal que $\sigma' \models q$.
 - Si desde σ_0 se ejecuta S y no alcanza un estado σ'_0 dentro de q entonces se cumple $\{p\}S\{q\}$ porque si $\sigma \not\models q$, $\{p\}S\{q\}$ es trivialmente verdadero.

Ejemplo de verificación axiomática de un programa

- Sea S_{fac} el programa que calcula el factorial de un número.
- El factorial $x!$ de un número $x > 0$ es $x! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot x$. Por ejemplo, $4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$.
- El programa S_{fac} es el siguiente:

S_fac::

```
a := 1;
y := 1;
while a < x do
  a := a + 1;
  y := y * a;
od
```

- Ejemplo de ejecución:

```
{x = 4}
a = 1
y = 1
```

```
1 < 4: a = 2, y = 1 * 2 = 2
2 < 4: a = 3, y = 2 * 3 = 6
3 < 4: a = 4, y = 6 * 4 = 24
```

- Se quiere verificar $\{x > 0\} S_{fac} \{y = x!\}$. Es decir, hay que probar que desde todo estado $\sigma \models x > 0$, S_{fac} termina en un estado σ' tal que $\sigma' \models y = x!$:
 - Si $x = 1$, luego de ejecutar S_{fac} debe valer $y = 1$.
 - Si $x = 2$, luego de ejecutar S_{fac} debe valer $y = 2$.
 - Si $x = 3$, luego de ejecutar S_{fac} debe valer $y = 6$.
 - Si $x = 4$, luego de ejecutar S_{fac} debe valer $y = 24$.
 - ...
- Notar que si $x \leq 0$ queda $y = 1 \neq x!$, cosa que no es correcta. Sin embargo, como la precondition es $x > 0$, el programa es correcto con respecto a la especificación dada.

Componentes del método de verificación axiomática (Lógica de Hoare)

1. Lenguaje de programación (programas con while):

- Instrucciones:
 - $S :: x := e$
 - $S :: S_1; S_2$
 - $S :: \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}$
 - $S :: \text{while } B \text{ do } S_1 \text{ od}$
- Expresiones de tipo entero:
 - $e :: n$
 - $e :: x$
 - $e :: (e_1 + e_2)$
 - $e :: (e_1 - e_2)$
 - $e :: (e_1 \cdot e_2)$
 - Donde n es una constante entera y x es una variable entera.

- Expresiones de tipo booleano:

- $B :: true$
- $B :: false$
- $B :: (e_1 = e_2)$
- $B :: (e_1 < e_2)$
- ...
- $B :: \neg B_1$
- $B :: (B_1 \vee B_2)$
- $B :: (B_1 \wedge B_2)$
- ...

2. Lenguaje de especificación (lógica de predicados):

- $p :: true$
- $p :: false$
- $p :: (e_1 = e_2)$
- $p :: (e_1 < e_2)$
- ...
- $p :: \neg p$
- $p :: (p_1 \vee p_2)$
- $p :: (p_1 \wedge p_2)$
- ...
- $p :: \exists x : p$
- $p :: \forall x : p$

Axiomática de la lógica de Hoare

Axioma de la asignación (ASI)

- $\{p[x|e]\}x := e\{p\}$
- Se lee de derecha a izquierda: si luego de ejecutar $x := e$ vale p en términos de x , entonces antes de ejecutar $x := e$ valía p en términos de e .
- Por ejemplo:
 - $\{?\}x := x + 1\{x \geq 0\}$
 - $\{x \geq 0[x|x+1]\}x := x + 1\{x \geq 0\}$
 - $\{x + 1 \geq 0\}x := x + 1\{x \geq 0\}$
 - Es decir, si luego de $x := x + 1$ vale $x \geq 0$, entonces antes de $x := x + 1$ valía $x + 1 \geq 0$.

Regla de la secuencia (SEC)

- $$\frac{\{p\}S_1\{r\}, \{r\}S_2\{q\}}{\{p\}S_1;S_2\{q\}}$$
- El predicado r actúa como un nexo entre S_1 y S_2 y luego se descarta, no se propaga.
- Se puede generalizar a n instrucciones.

Regla del condicional (COND)

- $$\frac{\{p \wedge B\}S_1\{q\}, \{p \wedge \neg B\}S_2\{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$$

- Formula un modo de verificar una selección condicional fijando un único punto de entrada y un único punto de salida, correspondientes a p y q respectivamente.

Regla de la repetición (REP)

- $$\frac{\{p \wedge B\} S \{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$
- p se define en términos de las variables del programa S .
- p es un predicado que vale antes y después de toda iteración, es decir, es un **invariante**.
- La regla NO asegura que el while termine.
- Si p vale al comienzo del bucle y mientras vale B el cuerpo S preserva p , entonces por razonamiento inductivo p vale al terminar el bucle.

Regla de la consecuencia (CONS)

- $$\frac{p \rightarrow p_1, \{p_1\} S \{q_1\}, q_1 \rightarrow q}{\{p\} S \{q\}}$$
- Se puede reemplazar p_1 por un predicado p que lo implique.
- Se puede reemplazar q_1 por un predicado q al que implique.
- Permite reforzar precondiciones y debilitar postcondiciones:
 - De $\{x > 0\} S \{x = 0\}$ y $x > 5 \rightarrow x > 0$ se deduce $\{x > 5\} S \{x = 0\}$.
 - De $\{true\} S \{x = y + 1\}$ y $x = y + 1 \rightarrow x > y$ se deduce $\{true\} S \{x > y\}$.
- La regla no depende del lenguaje de programación si no del dominio semántico. Es una regla semántica más que sintáctica.
- Permite usar todos los axiomas del dominio semántico, como pueden ser los números enteros.

Composicionalidad

- El método de prueba presentado es **composicional**: dado un programa S , compuesto por subprogramas $S_1, S_2, \dots, S_n$, que valga la fórmula $\{p\} S \{q\}$, depende únicamente de que valgan las fórmulas $\{p_1\} S_1 \{q_1\}$, $\{p_2\} S_2 \{q_2\}, \dots, \{p_n\} S_n \{q_n\}$, sin importar el contenido de los subprogramas S_i (noción de **caja negra**).
- Por ejemplo, dado el programa $S :: S_1; S_2$, si se cumplen las fórmulas $\{p\} S_1 \{r\}$ y $\{r\} S_2 \{q\}$, entonces también se cumple la fórmula $\{p\} S_1; S_2 \{q\}$, **independientemente del contenido de S_1 y S_2** .
- Si en vez de S_2 usamos un subprograma S_3 que también satisface $\{r\} S_3 \{q\}$, entonces también se cumple la fórmula $\{p\} S_1; S_3 \{q\}$, lo que implica que S_2 y S_3 son **intercambiables** (son funcionalmente equivalentes respecto de la tupla (r, q)).

Pérdida de la composicionalidad en los programas concurrentes

- En los programas concurrentes, la composicionalidad se pierde porque el orden de ejecución de los subprogramas no es fijo, sino que varía en cada ejecución. Por lo tanto, el resultado de ejecutar un subprograma puede depender del orden de ejecución de los subprogramas, lo que hace que no se puedan intercambiar subprogramas sin afectar el resultado final.
- Por ejemplo, si $\{x = 0\} S_1 :: x := x + 2 \{x = 2\}$ y $\{x = 0\} S_2 :: z := x \{z = 0\}$, y se ejecutan de forma concurrente, entonces no siempre se cumple que $\{x = 0 \wedge x = 0\} [S_1 \parallel S_2] \{x = 2 \wedge z = 0\}$, porque si en el programa $S_1 \parallel S_2$ se ejecuta primero S_1 y luego S_2 , al final se cumple que $z = 2$ en vez de $z = 0$.
 - Lo que SÍ vale es $\{x = 0 \wedge x = 0\} [S_1 \parallel S_2] \{x = 2 \wedge (z = 0 \vee z = 2)\}$

- Además, cambiar S_1 por un proceso equivalente $S_3 :: x := x + 1; x := x + 1$ no vale porque si S_2 se ejecuta entre medio de las dos asignaciones de S_3 , entonces al final se cumple que $z = 1$ en vez de $z = 0$ o $z = 2$.
Así, **en la concurrencia dos procesos funcionalmente equivalentes NO son intercambiables**.
- Se pierde la noción de caja negra.

Especificaciones

- La especificación de un programa para calcular el factorial: $\{x > 0\}S_{fac}\{y = x!\}$ no es correcta, por que si bien el programa $S :: x := 1; y := 1$ satisface $(x > 0, y = x!)$, no es el programa pedido. Por ejemplo, si al empezar $x = 5$, entonces al final debe ser $y = 5! = 120$ pero se tiene $y = 1$.
- El problema central es que las variables de la precondition pueden ser modificadas a lo largo del programa, efectivamente rompiendo la conexión entre la precondition y la postcondition. Para solucionar esto se introduce el concepto de **variable lógica**.
- Una variable lógica es una variable que aparece en la precondition pero no en el programa, es decir, no es modificada por el programa. Se usan para congelar valores.
- En el ejemplo dado, haríamos $(x = X \wedge X > 0, y = X!)$ con X la variable lógica.
- No es posible agregar a la especificación que la variable común x no se modifique nunca, ya que la lógica de predicados no lo permite. Una lógica que sí lo permite es la lógica temporal.

Sensatez, completitud y automatización de la verificación

- La **sensatez** es la propiedad obligatoria de que si una fórmula $\{p\}S\{q\}$ se cumple sintácticamente (con axiomas y reglas) entonces también se cumple semánticamente (considerando estados y computaciones).
- La **completitud** es la propiedad deseable de que si una fórmula $\{p\}S\{q\}$ se cumple semánticamente entonces se puede probar sintácticamente.
 - Su cumplimiento o no depende de la expresividad del lenguaje de especificación.
- La **automatización** no es posible porque la verificación de programas es indecidible en general y por lo tanto no automatizable. Sin embargo, existen muchas herramientas que asisten interactivamente al programador en la verificación de programas. En particular, si los programas son de estados finitos, existen sistemas de verificación automática denominados sistemas de model checking, que usan la lógica temporal y no son sintácticamente orientados como la Lógica de Hoare.
 - Herramientas de verificación axiomática: Dafny, COQ, Isabelle.
 - Herramientas de model checking: EMC, CAESAR, SMV, SPIN.
 - Herramientas de síntesis de programas: búsqueda estocástica, IA.

Clase 9 - 26 de mayo, 2026

Lógica de Hoare

Ejemplo de prueba 1

- Se tiene un programa que intercambia los valores de dos variables.
- Dado $S_{swap} :: z := x; x := y; y := z$, se va a probar sintácticamente $\{x = X \wedge y = Y\} S_{swap} \{y = X \wedge x = Y\}$ con X e Y variables lógicas.
- Pasos:
 - i. $\{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\}$ (ASI)
 - ii. $\{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\}$ (ASI)
 - iii. $\{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\}$ (ASI)
 - iv. $\{x = X \wedge y = Y\} z := x; x := y; y := z \{y = X \wedge x = Y\}$ (1, 2, 3, SEC)
- Por la sensatez del método, como se probó sintácticamente $\{x = X \wedge y = Y\} S_{swap} \{y = X \wedge x = Y\}$, entonces también se cumple semánticamente.
- Para probar la fórmula similar $\{y = Y \wedge x = X\} S_{swap} \{y = X \wedge x = Y\}$ sintácticamente se agregan dos pasos luego del paso 4:
 - i. $(y = Y \wedge x = X) \rightarrow (x = X \wedge y = Y)$ (MAT)
 - ii. $\{y = Y \wedge x = X\} S_{swap} \{y = X \wedge x = Y\}$ (4, 5, CONS)

Ejemplo de prueba 2

- Se tiene un programa que calcula el valor absoluto de una variable.
- Dado $S_{abs} :: \text{if } x > 0 \text{ then } y := x \text{ else } y := -x \text{ fi}$, se va a probar sintácticamente $\{true\} S_{abs} \{y \geq 0\}$
- Como tenemos un if, hay que usar la regla COND.
- Para llegar a $\{true\} S_{abs} \{y \geq 0\}$ por COND, hay que obtener $\{true \wedge x > 0\} y := x \{y \geq 0\}$ y $\{true \wedge \neg(x > 0)\} y := -x \{y \geq 0\}$.
- Considerando las dos asignaciones del programa, los primeros pasos de la prueba podrían ser:
 - i. $\{x \geq 0\} y := x \{y \geq 0\}$ (ASI)
 - ii. $\{-x \geq 0\} y := -x \{y \geq 0\}$ (ASI)
- Luego podemos combinar el axioma ASI con la regla COND para continuar:
 - i. $(true \wedge x > 0) \rightarrow (x \geq 0)$ (MAT)
 - ii. $\{true \wedge x > 0\} y := x \{y \geq 0\}$ (1, 3, CONS)
 - iii. $(true \wedge \neg(x > 0)) \rightarrow (-x \geq 0)$ (MAT)
 - iv. $\{true \wedge \neg(x > 0)\} y := -x \{y \geq 0\}$ (2, 5, CONS)
 - v. $\{true\} \text{if } x > 0 \text{ then } y := x \text{ else } y := -x \text{ fi} \{y \geq 0\}$ (4, 6, COND)

Axiomática para la correctitud total (Regla REP*)

- La regla REP solo permite probar la invariancia de un predicado p .

- La regla nueva REP* que se introducirá a continuación permite probar además la terminación del while. Se basa en un predicado invariante p y una función variante t que es entera y está definida, como el invariante, en términos de las variables del programa.
- Regla REP*:
$$\frac{\langle p \wedge B \rangle S \langle p \rangle, \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$$
- Se le agregan dos premisas a REP y se reemplazan las llaves por corchetes angulares.
- En la segunda premisa, t se decrementa en cada iteración.
- Z es una variable lógica, no aparece en p , B , t ni en S .
- El objetivo de Z es fijar el valor de t antes de ejecutar S .
- En la tercera premisa, t se mantiene menor o igual a cero a lo largo de todo el bucle.
- Así, el bucle debe necesariamente terminar, porque en los números naturales no hay cadenas descendentes infinitas.

Separación de la prueba del while en dos

- Si bien la regla REP* permite probar tanto la correctitud parcial como la terminación de un while, es recomendable separar la prueba en dos partes:
 - Primero se prueba $\{p\}S\{q\}$ con REP para asegurar la invariancia de p .
 - Luego se prueba $\langle p \rangle S \langle true \rangle$ con REP* para asegurar la terminación del bucle.
 - La primera prueba se lee: "Si S termina a partir de p , lo hace en q ".
 - La segunda prueba se lee: " S termina a partir de p ".
 - Las dos pruebas puestas juntas se leen: " S termina a partir de p y lo hace en q ".

Clase 10 - 2 de junio, 2026

Sensatez y completitud de H y H^*

Sensatez de H

No puede pasar que **SÍ se pueda demostrar** en H , por ejemplo, $\{x = 0\}x := x + 1\{x = 2\}$, porque ocurre que la precondition es verdadera, pero luego de ejecutar el programa no se cumple la postcondición.

Completitud de H

No puede pasar que **NO se pueda demostrar** en H , por ejemplo, $\{x = 0\}y := x\{y = 0\}$, porque ocurre que la precondition es verdadera y luego de ejecutar el programa se cumple la postcondición, por lo tanto la terna es válida.

Sensatez de H^*

No puede pasar que **SÍ se pueda demostrar** en H^* , por ejemplo, $\langle x = 0 \rangle \text{ while } x = 0 \text{ do } skip \text{ od } \langle true \rangle$, porque ocurre que la precondition es verdadera, pero el programa no termina.

Completitud de H^*

No puede pasar que **NO se pueda demostrar** en H^* , por ejemplo, $\langle x = 0 \rangle \text{ while } x < 10 \text{ do } x := x + 1 \text{ od } \langle x = 10 \rangle$,

Simbología de la sensatez y completitud

- $\vdash_H \{p\}S\{q\}$ expresa que en el sistema H se puede demostrar la fórmula $\{p\}S\{q\}$.
- $\models \{p\}S\{q\}$ expresa que la fórmula $\{p\}S\{q\}$ se cumple **semánticamente**.
- $\vdash_{H^*} \langle p \rangle S \langle q \rangle$ expresa que en el sistema H^* se puede demostrar la fórmula $\langle p \rangle S \langle q \rangle$.
- $\models \langle p \rangle S \langle q \rangle$ expresa que la fórmula $\langle p \rangle S \langle q \rangle$ se cumple **semánticamente**.
- Se prueba que:
 - Si $\vdash_H \{p\}S\{q\}$ entonces $\models \{p\}S\{q\}$ (**sensatez** de H).
 - Si $\models \{p\}S\{q\}$ entonces $\vdash_H \{p\}S\{q\}$ (**completitud** de H).
 - Si $\vdash_{H^*} \langle p \rangle S \langle q \rangle$ entonces $\models \langle p \rangle S \langle q \rangle$ (**sensatez** de H^*).
 - Si $\models \langle p \rangle S \langle q \rangle$ entonces $\vdash_{H^*} \langle p \rangle S \langle q \rangle$ (**completitud** de H^*).
- En términos simples:
 - Las fórmulas que demuestran los sistemas H y H^* son verdaderas $\rightarrow$ **Sensatez**.
 - Todas las fórmulas que son verdaderas se pueden demostrar en los sistemas H y H^* $\rightarrow$ **Completitud**.

Inducción matemática

- Sea P una propiedad que se quiere probar en $\mathbb{N}$.
- Se compone de 3 pasos fundamentales: base inductiva, hipótesis inductiva y paso inductivo.
- **Base inductiva:** Si se cumple que $P(0)$ es verdadera (donde 0 es el elemento mínimo de $\mathbb{N}$)
- **Hipótesis inductiva:** Supongamos que $P(k)$ es verdadera para algún $k \in \mathbb{N}$.
- **Paso inductivo:** Y además que $\forall k \in \mathbb{N}, P(k) \rightarrow P(k+1)$, entonces se concluye que para todo $n = 0, 1, 2, \dots$, $P(n)$ es verdadera.
- Ejemplo de demostración de la sumatoria de n números naturales usando inducción matemática:
 - $\sum_{i=0}^n i = \frac{n(n+1)}{2}$
 - **Base inductiva:** $P(0) : \sum_{i=0}^0 i = \frac{0(0+1)}{2}$, es decir, $0 = 0$, lo cual es verdadero.
 - **Hipótesis inductiva:** Supongamos que $P(k) : \sum_{i=0}^k i = \frac{k(k+1)}{2}$ es verdadera para algún $k \in \mathbb{N}$.
 - **Paso inductivo:** Entonces, $\sum_{i=0}^{k+1} i = \sum_{i=0}^k i + (k+1)$. Por hipótesis inductiva, esto es igual a $\frac{k(k+1)}{2} + (k+1)$. Factorizando, se obtiene $\frac{k(k+1)+2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$, lo cual es igual a $\frac{(k+1)((k+1)+1)}{2}$, y por lo tanto se cumple que $\forall k \in \mathbb{N}, P(k) \rightarrow P(k+1)$.

Inducción estructural

- Es más general que la inducción matemática.
- Se usa para definir conjuntos y demostrar sus propiedades.
- Se aplica a cualquier dominio (los naturales, expresiones aritméticas, árboles, programas, etc).
- Puede haber varias bases inductivas y varios pasos inductivos.

- Ejemplo:
 - Definir por I.E las expresiones aritméticas con símbolos del conjunto $\{0, 1, x, +, \cdot, (,)\}$
 - **Bases inductivas:** 0 , 1 y x son expresiones aritméticas.
 - **Pasos inductivos:** si e y f son expresiones aritméticas, entonces $(e + f)$ y $(e \cdot f)$ también lo son.
 - Probar que $C(e) = 1 + O(e)$, donde C es la cantidad de 0 , 1 y x , y O es la cantidad de operadores $+$ y $\cdot$.
 - **Bases inductivas:**
 - $C(0) = 1 + O(0)$
 - $C(1) = 1 + O(1)$
 - $C(x) = 1 + O(x)$
 - **Paso inductivo para el operador $+$:**
 - Supongamos que $C(e) = 1 + O(e)$ y $C(f) = 1 + O(f)$.
 - Entonces, $C(e + f) = C(e) + C(f) = (1 + O(e)) + (1 + O(f))$ (hipótesis)
 - Como $O(e + f) = O(e) + 1 + O(f)$, entonces $C(e + f) = 1 + O(e + f)$.

Semántica del lenguaje de programación

Conceptos básicos

- Para probar las propiedades de sensatez y completitud de H y H^* , primero hay que definir formalmente la semántica del lenguaje de programación.
- Dados un programa S y un estado inicial σ , a dicha instancia o configuración inicial (S, σ) se le asocia una computación $\pi(S, \sigma)$ que es la secuencia de instancias de la ejecución de S a partir de σ .
 - Por ejemplo: $(x := 0; y := 1; z := 2, \sigma) \rightarrow (y := 1; z := 2, \sigma[x|0]) \rightarrow (z := 2, \sigma[x|0][y|1]) \rightarrow (E, \sigma[x|0][y|1][z|2])$.
- $val(\pi(S, \sigma))$ es el estado final de la computación $\pi(S, \sigma)$. Si $\pi(S, \sigma)$ es infinita, entonces $val(\pi(S, \sigma)) = \perp$ (no se detiene).
 - Ejemplo 1: $val(\pi(\text{while true do skip od}, \sigma)) = \perp$.
 - Ejemplo 2: $val(\pi(x := 10; \text{while } x > 0 \text{ do } x := x - 1 \text{ od}, \sigma)) = \sigma[x|0]$.

Definición por inducción estructural

- El lenguaje de programación se define por inducción estructural mediante una relación $\rightarrow$:
- $(skip, \sigma) \rightarrow (E, \sigma)$
 - El *skip* se consume en un paso (es atómico) y no modifica el estado inicial. E es la continuación sintáctica vacía.
- $(x := e, \sigma) \rightarrow (E, \sigma[x|\sigma(e)])$
 - La asignación $x := e$ es atómica y el estado final es como el inicial salvo que el valor de x es el de la expresión e según σ .
- Para toda instrucción T , si $(S, \sigma) \rightarrow (S', \sigma')$ entonces $(S; T, \sigma) \rightarrow (S'; T, \sigma')$
 - La secuencia $S; T$ se ejecuta de izquierda a derecha. Consumido S , si no diverge (si no loopea), se ejecuta T .
- $(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma) \rightarrow (S_1, \sigma)$ si se cumple $\sigma(B) = \text{verdadero}$
- $(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma) \rightarrow (S_2, \sigma)$ si se cumple $\sigma(B) = \text{falso}$
 - La evaluación de la expresión B es atómica y no modifica el estado inicial. Su evaluación establece si se ejecuta S_1 o S_2 .

- $(\text{while } B \text{ do } S \text{ od}, \sigma) \rightarrow (S; \text{while } B \text{ do } S \text{ od}, \sigma)$ si se cumple $\sigma(B) = \text{verdadero}$
- $(\text{while } B \text{ do } S \text{ od}, \sigma) \rightarrow (E, \sigma)$ si se cumple $\sigma(B) = \text{falso}$
 - La evaluación de la expresión B es atómica y no modifica el estado inicial. Su evaluación establece si se ejecuta el cuerpo S o si se termina el bucle.

Nota

- Las expresiones $\sigma(e)$ y $\sigma(B)$ son abreviaciones, porque un estado σ se define sobre variables y no sobre expresiones.
- Por ejemplo, si $e :: (x + 8)$, entonces $\sigma(e) = \sigma(x) + \sigma(8)$.

Definición por extensión de la semántica de las instrucciones

- De la definición por comprensión o inductiva de las instrucciones se puede derivar su definición por extensión.
- Por ejemplo, en la secuencia, una computación $\pi(S_1; S_2, \sigma)$ tiene tres formas posibles:
 - Una computación **infinita** $(S_1; S_2, \sigma_0) \rightarrow (T_1; S_2, \sigma_1) \rightarrow (T_2; S_2, \sigma_2) \rightarrow \dots$ cuando S_1 diverge a partir de σ_0 .
 - Una computación **infinita** $(S_1; S_2, \sigma_0) \rightarrow \dots \rightarrow (S_2, \sigma_1) \rightarrow (T_1, \sigma_2) \rightarrow (T_2, \sigma_3) \rightarrow \dots$ cuando S_1 termina a partir de σ_0 pero S_2 diverge a partir de σ_1 .
 - Una computación **finita** $(S_1; S_2, \sigma_0) \rightarrow \dots \rightarrow (S_2, \sigma_1) \rightarrow \dots \rightarrow (E, \sigma_2)$ cuando S_1 termina a partir de σ_0 y S_2 termina a partir de σ_1 .
- De forma similar se pueden obtener las formas de las computaciones de los otros tipos de instrucciones.

Prueba de la sensatez del método H

- Se cumple, para todo programa S y toda especificación (p, q) que $\vdash_H \{p\}S\{q\} \rightarrow \models \{p\}S\{q\}$.
- Se prueba por inducción matemática fuerte, considerando la longitud de la prueba (1 o más pasos):
 - Base inductiva: los axiomas son verdaderos (pruebas de tamaño 1).
 - Paso inductivo: las reglas son sensatas, es decir, preservan la verdad de las premisas (pruebas de tamaño ≥ 2)
- Ejemplo 1: Prueba de que el axioma *skip* es verdadero.
 - Dado $\vdash \{p\}\text{skip}\{p\}$, se quiere probar $\models \{p\}\text{skip}\{p\}$.
 - Sea $\vdash \{p\}\text{skip}\{p\}$.
 - Sea $\sigma \models p$.
 - La semántica del *skip* es $(\text{skip}, \sigma) \rightarrow (E, \sigma)$.
 - Así, luego de un *skip* queda $\sigma \models p$.
 - Por lo tanto, $\models \{p\}\text{skip}\{p\}$.
- Ejemplo 2: Prueba de que la regla SEC es sensata (preserva la verdad de las premisas)
 - Dado $\vdash \{p\}S_1; S_2\{q\}$ hay que probar $\models \{p\}S_1; S_2\{q\}$.
 - Sea $\vdash \{p\}S_1; S_2\{q\}$.
 - Se cumple $\vdash \{p\}S_1\{r\}$ y $\vdash \{r\}S_2\{q\}$ para algún r .
 - Las pruebas $\vdash \{p\}S_1\{r\}$ y $\vdash \{r\}S_2\{q\}$ son más cortas que la prueba $\vdash \{p\}S_1; S_2\{q\}$, por lo tanto se puede aplicar inducción.
 - Por hipótesis inductiva: $\models \{p\}S_1\{r\}$ y $\models \{r\}S_2\{q\}$.

- Sea $\sigma_1 \models p$.
- La semántica de la secuencia es $(S_1; S_2, \sigma_1) \rightarrow \dots \rightarrow (S_2, \sigma_2) \rightarrow \dots \rightarrow (E, \sigma_3)$.
- Así, como $\models \{p\}S_1\{r\}$ y $\models \{r\}S_2\{q\}$, entonces luego de $S_1; S_2$ queda $\sigma_3 \models q$.
- Por lo tanto $\models \{p\}S_1; S_2\{q\}$.

Prueba de la sensatez del método H^*

- Solo hay que probar que la regla REP* es sensata.
- $\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle \rightarrow \models \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$.
- Sea $\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$.
- Se cumplen dos cosas:
 - $\vdash \langle p \wedge B \rangle S \langle p \rangle$
 - $\vdash \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0$
- Las pruebas $\vdash \langle p \wedge B \rangle S \langle p \rangle$ y $\vdash \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0$ son más cortas que la prueba $\vdash_{H^*} \langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$, por lo tanto se puede aplicar inducción.
- Por hipótesis inductiva: $\models \langle p \wedge B \rangle S \langle p \rangle$ y $\models \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0$.
- Sea $\sigma_0 \models p$. Hay que probar que el while termina en un estado $\sigma_k \models \langle p \wedge \neg B \rangle$.
- Supongamos que el while loopea a partir de σ_0 .
- Así, se tiene la computación infinita $(\text{while } B \text{ do } S \text{ od}, \sigma_0) \rightarrow (\text{while } B \text{ do } S \text{ od}, \sigma_1) \rightarrow (\text{while } B \text{ do } S \text{ od}, \sigma_2) \rightarrow \dots$ y por hipótesis inductiva, siempre vale $p, t \geq 0$ y t se decrementa en cada iteración.
- Pero esto no puede pasar, porque si no habría una cadena descendente infinita de números naturales.
- Por lo tanto el while termina y lo hace en algún estado $\sigma_k \models \langle p \wedge \neg B \rangle$